



MAMHSAN: A multi-agent deep reinforcement learning framework based on multi-head self-attention network with heterogeneous graph embedding for flexible job shop scheduling[☆]

Zi-Qi Zhang^{a,b,c,1,*}, Tong Huang^{a,b,1}, Bin Qian^{a,b,c,**},
Rong Hu^{a,b}

^a School of Information Engineering and Automation, Kunming University of Science and Technology, Kunming 650500, China

^b The Higher Educational Key Laboratory for Industrial Intelligence and Systems of Yunnan Province, Kunming University of Science and Technology, Kunming 650500, China

^c Yunnan Key Laboratory of Artificial Intelligence, Kunming University of Science and Technology, Kunming 650500, China

ARTICLE INFO

Keywords:

Flexible job shop scheduling
Multi-agent reinforcement learning
Deep reinforcement learning
Heterogeneous graph
Multi-head self-attention network

ABSTRACT

Industrial intelligence serves as a crucial role in advancing smart manufacturing, with flexible manufacturing systems (FMS) emerging as a predominant production paradigm. This article introduces a novel multi-agent deep reinforcement learning framework to tackle the flexible job shop scheduling problem (FJSP). To overcome the limitations in existing Markov decision process (MDP)-based models for FJSPs, we introduce a novel multi-dimensional action and reward MDP (MD-MDP) formulation. Building upon this foundation, we develop an innovative multi-agent reinforcement learning framework (MAMHSAN) that integrates multi-head self-attention network (MHSAN) with the heterogeneous graph embedding used to solve FJSPs. In MAMHSAN, agents independently make decisions in multidimensional actions, tackling challenges related to machine selection and operation sequencing. The integration of the MHSAN with the heterogeneous graph representation significantly improves feature extraction efficacy and decision-making efficiency by capturing critical correlations among operations and machines. Furthermore, a multi-agent proximal policy optimization (MA-PPO) algorithm is proposed for trial-and-error training and feedback optimizing parameters of MAMHSAN, further enhancing collaborative decision-making capabilities for agents. Extensive experiments and comprehensive comparisons demonstrate MAMHSAN's effectiveness and computational efficiency, with statistical analyses demonstrating the efficacy of each critical component of our method and the significant superiority of the proposed MAMHSAN over state-of-the-art algorithms for solving FJSPs.

1. Introduction

Industrial intelligence-driven smart manufacturing is transforming modern manufacturing systems, where industrial IoT-enhanced flexible manufacturing systems (FMS), through their capabilities for real-time responsiveness and dynamic adaptability, are emerging as a pivotal paradigm for achieving intelligent scheduling and collaborative decision-making (Yang et al., 2021). Such advancements bridge the gap between massive production and customized production, significantly enhancing the agility of enterprises to respond to changes in market

dynamics. A critical challenge in implementing FMS is to address the flexible job shop scheduling problem (FJSP), a well-known NP-hard problem (Xie et al., 2019) that remains a central research focus in intelligent scheduling research. The FJSP involves selecting appropriate machines for operations and determining superior sequences for these operations, thereby significantly increasing its complexity (Li et al., 2022; Xie et al., 2019). This critical challenge is commonly encountered across industries including semiconductor fabrication, petrochemical sectors, and motive manufacturing (Wang & Zhang, 2023; Wu et al., 2022; Zhang et al., 2023b; Zhang et al., 2025). Due to the potential for

[☆] This article is part of a special issue entitled: 'AI-based Approaches' published in Computers & Industrial Engineering.

* Corresponding author at: School of Information Engineering and Automation, Kunming University of Science and Technology, Kunming 650500, China.

** Corresponding author at: School of Information Engineering and Automation, Kunming University of Science and Technology, Kunming 650500, China.

E-mail addresses: zhangziqui@kust.edu.cn (Z.-Q. Zhang), huangtong@stu.kust.edu.cn (T. Huang), bin.qian@kust.edu.cn (B. Qian), ronghu_da@kust.edu.cn (R. Hu).

¹ The authors contributed equally to this work.

significant applications in computer-integrated manufacturing and industrial engineering, studies on FJSPs and their solving methods are of great interest in both academic and industrial domains.

Previous studies have classified methods for solving FJSPs into exact and approximate approaches. Among the exact methods, mathematical programming represents the most representative technique (Meng et al., 2019), accurately modeling the problem and systematically exploring the solution space through rigorous mathematical formulations. These MILP methods guarantee optimal solutions for small-scale instances but suffer from a combinatorial explosion when dealing with large-scale FJSPs due to inherent complexity, rendering high-quality and timely solutions infeasible (Meng et al., 2020). Consequently, approximate methods are predominantly adopted for practical applications, including heuristic and metaheuristic algorithms. Heuristic algorithms quickly generate approximate solutions by formulating rules and search strategies based on job/operation prioritization, yet their randomness may compromise solution optimality. Metaheuristic algorithms, conversely, iteratively refine initial solutions toward high-quality optima. Despite their efficacy, these approaches demand considerable computational costs and resources, exhibit limited generalization capabilities for large-scale FJSPs, and are prone to premature convergence at local optima (Sharma & Tripathi, 2022). Thus, developing effective and efficient approaches that balance solution quality with computational efficiency persists as an unresolved challenge in addressing FJSPs.

Recent research has witnessed that deep reinforcement learning (DRL) has emerged as a promising learning-based framework within intelligent optimal scheduling domains. DRL-based methods have demonstrated significant effectiveness in solving complex optimization problems and have also been widely applied in addressing FJSPs (Cunha et al., 2018). These techniques involve modeling the scheduling process of FJSPs as a Markov decision process (MDP), which generates superior solutions from end to end by treating state features as inputs and determining appropriate actions through agents. Notably, DRL-based methods for FJSPs learn effective policies for decisions directly from problem-specific features without predefining priority dispatching rules (PDRs). Upon integrating deep neural networks (DNNs) with RL techniques, DRL-based methods have shown strong strengths in solving various extensions of FJSPs (Zhao et al., 2021). Existing works reveal that the critical aspect of DRL-based methods lies in the inputs from state representations that consist of state features designed based on problem-specific knowledge (Liu et al., 2022; Luo, 2020). However, as the complexity of the problem increases, the finite number of scheduling states and feature representations are inadequate for comprehensively capturing environmental dynamics. As a result, the feature representations and feedback rewards between agents from interactions always suffer from trial-and-error experiences, which compromises policy adaptability and learning effects in dynamic environments.

Recent advancements have motivated some scholars to dynamically capture both scheduling states and feature representations in MDPs by graph-based modeling. Among these approaches, disjunctive graph and heterogeneous graph formulations have emerged as notable techniques for representing the feature states at various time steps in MDPs for FJSPs, with the introduction of these states into graph neural networks (GNNs) (Wu et al., 2021) for extracting problem-specific characteristics. This allows efficient representation of complex relationships between operations and machines through nodes and directed arcs, capturing rich potential information from the environment for agents to make decisions. Heterogeneous graphs introduce machine nodes with undirected arcs to illustrate inherent connections among operations and machines, greatly reducing the complexity of graph structures and providing an intuitive allocation relationship between operations and machines compared to disjunctive graphs. However, due to the complexity of FJSPs, disjunctive and heterogeneous graphs exhibit exponential growth in the density of graphs as scheduling process proceeds or problem scale increases. This leads to a sharp increase in the computational cost of GNN-based feature extraction when extracting

graph features in MDPs. Furthermore, in practical scenarios, the scale of FJSP varies, making it challenging for GNNs optimized for specific scales to adapt to different sizes of FJSPs, thus resulting in limited generalization ability of final solutions' performance (Munikoti et al., 2024). Consequently, designing effective GNNs to extract problem-specific characteristics, balancing computational efficiency with scale adaptability, and handling different scales of FJSPs, remains a significant research challenge.

Motivated by these observations, we propose an innovative multi-agent DRL framework based on multi-head self-attention network (MHSA) and the heterogeneous graph embedding (MAMHSAN) technique for solving the FJSP. In MAMHSAN, we innovatively integrate the heterogeneous graph representation with the multi-head self-attention network (MHSA) via feature embedding technique, which enables efficient cross-scale feature extraction through concatenation operations. Furthermore, a novel multi-dimensional action-reward MDP (MD-MDP) formulation is introduced to overcome limitations of conventional modeling methods. Based on MD-MDP, MAMHSAN uses both decision-time transfer (DDT) and scheduling action selection (SAS) agents to make collaborative decisions: the DDT agent is responsible for determining optimal decision time transfer, thus enabling the SAS agent to always make the best-fit choice in potential action spaces. Under such joint decision-making enabled by MD-MDP modeling, the scheduling solution quality of FJSP is significantly improved. The notable contributions can be summarized as follows:

- A novel MD-MDP modeling paradigm is introduced for FJSP, which shows strengths in terms of action space selection efficiency and state feature extraction effectiveness against previous DRL-based methods. In MD-MDP, comprehensive action space consideration, reasonable reward mechanisms, and state transitions enable MAMHSAN to have robust decision-making capabilities, thereby improving the performance of MD-MDP for solving FJSPs.
- The framework of multi-agent MHSA (MAMHSAN) with heterogeneous graph embedding is proposed for FJSP. In MAMHSAN, dual DDT and SAS agents derive the optimal actions through interactive feedback and collaboration, thus achieving efficient joint decision-making. Meanwhile, fine-grained reward shaping and experience sharing fasten training convergence while preserving policy optimality.
- A novel heterogeneous GNN (HGNN) based on a heterogeneous graph embedding technique with the MHSA is developed, providing a promising perspective for state representation and feature extraction in MDPs for tackling FJSPs. These designed state features are well-suited to the characteristics of FJSPs, enabling agents to effectively capture critical scheduling states and enhance decision-making efficiency.
- An improved multi-agent proximal policy optimization (MA-PPO) algorithm is proposed for training MAMHSAN. Based on the rewards received by DDT and SAS agents for performing actions, MA-PPO is effectively optimized for MAMHSAN, exhibiting the effectiveness and robust stability on validation benchmarks in computational experiments.
- Computational comparisons and comprehensive evaluations across four benchmark datasets confirm MAMHSAN's statistically significant superiority over both the composite priority dispatching rules (PDRs) and the state-of-the-art approaches in diverse scales and scenarios.

The rest of this article is organized as follows. In Section 2, related works are reviewed, focusing on current advances of approximation techniques for solving FJSPs. Section 3 provides an overview of the formulation and representation of heterogeneous graphs related to the FJSP. In Section 4, we evaluate existing modeling approaches categorized as MDPs for FJSPs, and introduce an innovative MD-MDP modeling method, along with state embedding of MHSA based on

heterogeneous graphs, a hierarchical decision-making framework of MAMHSAN, and training details of the MA-PPO algorithm. Comprehensive experimental results and statistical analyses across different benchmark instances and scenarios are presented in Section 5. Section 6 outlines insightful findings, conclusions, and suggestions for future research.

2. Literature review

2.1. Related work on FJSPs

Research on solution approaches for the FJSP involves tackling two primary subproblems: 1) identifying eligible processing machines for each operation of jobs, and 2) determining the processing orders of operations on machines. Unlike the classic JSP, the FJSP relaxes the constraints on machines, allowing each operation of jobs to be processed on multiple machines, thereby making the FJSP more complex and challenging. Existing methods for solving the FJSP and its variants are mainly classified into two categories: exact methods and approximate methods (Xie et al., 2019). Exact algorithms involve the mathematical modeling of the FJSPs, including mixed-integer linear programming (MILP) (Meng et al., 2019), and employ mathematical programming methods to find optimal solutions. Demir and Kürşat İşleyen (2013) proposed five mathematical models aimed at modeling FJSP and presented the MILP model to minimize the makespan. Gran et al. (2015) developed an MILP model designed to minimize the makespan and the total processing time, and this model demonstrated effectiveness for small-scale instances of FJSP. Meng et al. (2019) investigated MILP models for energy-efficient FJSP, implementing a turning off/on strategy to reduce idle energy consumption. They introduced six MILP models and conducted a comprehensive comparison with existing approaches, showing the superiority of their formulations. Naderi and Roshanaei (2022) exploited the decomposable structural features and developed an efficient constraint programming logic-based Benders decomposition (CP-LBBD) technique that integrates the strengths of MILP and CP models. While these exact algorithms are designed to discover optimal solutions by thoroughly examining candidate solutions and achieve excellent results for small-scale problems, their computational complexity escalates exponentially as the problem size increases, limiting their applicability (Meng et al., 2020).

In contrast, approximation methods include heuristics and metaheuristics, with emerging efforts that have confirmed these approaches can obtain satisfactory solutions. For solving FJSPs, heuristics commonly utilize composite PDRs which specific rules are formulated for the sequencing of operations and the allocation of machines, enabling the construction of feasible solutions. Regarding the PDRs for sequencing operations (Chiang & Fu, 2007; Doh et al., 2013; Narassima et al., 2022; Sobeyko & Mönch, 2016; Thenarasu et al., 2022), these rules include the most remaining operation priority rule (MOPNR), most workload priority rule (MWKR), and first-in-first-out rule (FIFO). Regarding the PDRs for machine allocation (Doh et al., 2013), usually used rules such as the shortest processing time (SPT), the longest processing time (LPT), and the earliest end time (EET). Although heuristic techniques can rapidly generate near-optimal solutions, they primarily depend on predefined priority rules and search strategies; thus, they exhibit inherent randomness and adaptability but offer no guarantee of optimality. The primary strength of these heuristics lies in their computational efficiency (Haupt, 1989; Sobeyko & Mönch, 2016; Thenarasu et al., 2022); however, their reliance on predefined priority rules, without sufficiently incorporating problem-specific knowledge and characteristics of FJSPs, often results in convergence to local optima. Metaheuristics derive suitable solutions by integrating diverse strategies and heuristics to iteratively improve the quality of solutions through evolutionary search mechanisms with problem-specific knowledge (Sharma & Tripathi, 2022; Zhang et al., 2024b). Rooyani and Defersha (2019) proposed a two-stage genetic algorithm (2SGA) for solving large-

scale instances of the FJSP. Caldeira and Gnanavelbabu (2019) devised an improved Jaya algorithm (IJA) incorporating effective initialization methods, local search strategies, and acceptance criteria to improve solution quality while preserving population diversity. Yang et al. (2022) introduced an improved Dragonfly algorithm incorporating a dynamic opposition-based learning (DOL) strategy for large-scale FJSP. Luo et al. (2023) proposed a Pareto-based two-stage evolutionary algorithm (PTEA) to efficiently address the FJSP with worker cooperation flexibility. Sun et al. (2023) proposed an improved hybrid genetic algorithm with variable neighborhood search (HGA-VNS) to tackle the FJSP, considering machine workload balancing while minimizing makespan. Yang et al. (2023) proposed a guided shuffled frog-leaping algorithm (GSFLA) to minimize the makespan of FJSP with variable sublots and overlapping operations. Xu et al. (2024) proposed a hybrid QPSO-VNS integrating quantum PSO (QPSO) and VNS to address FJSP. Han et al. (2024) proposed a dual population collaborative GA (DCGA) for the FJSP considering automatic guided vehicles (AGVs) transportation with the objective of minimizing makespan. Han and Gong (2025) devised a memetic algorithm based on non-dominated levels (MANL) for the FJSP with learn-forgetting effects and worker cooperation, aiming to minimize both makespan and worker workload. Although these metaheuristics are well-suited for addressing FJSPs with complex constraints across various scenarios, they typically require considerable computational costs for large-scale FJSPs and frequently converge to local optima with limited escape capabilities.

Recently, the successful application of DRL has introduced a promising perspective for addressing FJSPs (Panzer & Bender, 2022; Zhang et al., 2023c). By integrating the perceptual strengths of deep learning (LeCun et al., 2015) with the sequential decision-making framework of RL (Sutton & Barto, 2018), DRL-based methods have demonstrated significant efficacy in solving various combinatorial optimization problems (COPs). Compared to metaheuristics, DRL-based methods possess a notable advantage: the ability to dynamically adapt optimal actions (dispatching rules) at sequential decision points. These methods demonstrate enhanced effectiveness in reducing production costs, improving processing efficiency, and exhibiting superior flexibility and generalization capabilities (Li et al., 2023).

2.2. DRL-based methods for FJSPs

Recent years have witnessed significant research on learning-based methods for tackling FJSPs. Among these, DRL-based methods have emerged as prominent paradigms, effectively combining the strengths of DNNs and RL (Mnih et al., 2015). The demonstrated success of these methods across diverse domains has motivated extensive exploration into their application for solving various FJSPs (Cunha et al., 2020). DRL-based methods first formulate the FJSP as MDP, and then agents perceive information about production environments as state inputs by a DNN models that output priorities for feasible scheduling operations, thereby establishing end-to-end learning-based frameworks. Through training on production process datasets, the DNN model learns to adaptively select optimal actions in specific states to maximize total rewards relevant to optimization objectives. These methods focus on modeling FJSPs as MDPs and efficiently extracting feature states from environments, and scheduling strategies are learned directly from state inputs without relying on predefined PDRs. Moreover, DRL-based methods with well-trained DNN models deliver superior solutions across problem scales within acceptable timeframes and exhibit strong generalization capabilities (Lei et al., 2022; Song et al., 2023; Yuan et al., 2024). Due to advantages like rapid solving speed and strong generalization, several scholars are increasingly interested in solving FJSPs by applying DRL-based methods.

There are critical challenges in applying DRL-based methods to FJSPs, including effective MDP modeling, state representation, and feature extraction. Earlier efforts typically designed states based on process parameters and implemented with DRL frameworks to tackle

Table 1

Survey of DRL-based methods for solving FJSPs.

Author(s)	Graph representation	Embedding network	Objective	Problem	Description
Zeng et al. (2022)	Disjunction Graph	GNN	Makespan	FJSP	A DRL framework with GNN based on a disjunctive graph for FJSP.
Lei et al. (2022)	Disjunction Graph	MPGN	Makespan	FJSP	An end-to-end DRL framework with MPGN based on a disjunctive graph for FJSP.
Zhang et al. (2023a)	Disjunction Graph	GNN	Makespan	FJSP-VPT	A two-stage DRL dynamic scheduling framework with GNN based on a disjunctive graph for VPT-FJSP.
Lei et al. (2024)	Disjunction Graph	GNN	Makespan	DFJSP	A novel end-to-end hierarchical RL framework with GNN based on a disjunctive graph for DFJSP with random job arrivals.
Yuan et al. (2024)	Disjunction Graph	DNN	Makespan	FJSP	A DRL framework with a DNN based on a disjunctive graph for FJSP.
Zhang et al. (2024a)	Disjunction Graph	GAT, TE	Makespan	FJSP	A CARL framework with GAT and TE methods based on a disjunctive graph for FJSP.
Yuan et al. (2024)	Disjunction Graph	DAN	Makespan	FJSP	An end-to-end DRL framework with DAN methods based on a disjunctive graph for FJSP.
Song et al. (2023)	Heterogeneous Graph	HGNN	Makespan	FJSP	A novel DRL framework with HGNN based on heterogeneous graphs for FJSP.
Wan et al. (2024)	Heterogeneous Graph	MHGNN	Makespan	FJSP	An end-to-end DRL framework with MHGNN based on heterogeneous graphs for large-scale FJSP.
Wan et al. (2025)	Heterogeneous Graph	IGAT	Makespan	FJSP	A MAGRL framework with the IGAT method based on heterogeneous graphs for FJSP.
Ours	Heterogeneous Graph	MHSAN	Makespan	FJSP	A multi-agent DRL framework with MHSAN based on heterogeneous graphs for FJSP.

FJSPs. Luo (2020) proposed a deep Q-network (DQN) based method for dynamic FJSP (DFJSP) considering new job insertions, defining seven state features characterizing machine and job utilization, and delay rates to facilitate informed DQN decisions. Subsequently, Luo et al. (2022) introduced a hierarchical multi-agent DRL-based method for real-time scheduling of dynamic partial-no-wait multi-objective FJSP with new job insertions and machine breakdowns. They developed 20 features related to aspects such as machine and job utilization, and operation completion rate for effective DQN decision-making. Liu et al. (2022) presented a hierarchical distributed DRL-based method for FJSP with random job arrivals, in which three specific features are defined for distinct job sets to enable double DQN (DDQN) agent decisions. Liu et al. (2023) proposed a multi-agent DRL solver for dynamic FJSP (DFJSP), which designed five features for candidate jobs based on processing and waiting times, and fed these features into DDQN for near real-time scheduling decisions. While above studies achieved notable success for FJSPs, the generalization capability of these methods often requires researchers to have insights into problem-specific knowledge and domain expertise. The definitions of features are limited to specific aspects of jobs or machines and may not adequately capture the characteristics of the environments.

To capture high-quality state features from scheduling environments, recent interests increasingly tend to attempt feature extraction by GNNs (Wu et al., 2021). Significant efforts leverage disjunctive graph representations. Zhang et al. (2020) developed a DRL-based method that combines DRL and GNN for the JSP, modeling states as disjunctive graphs and encoding state graphs by GNNs. Park et al. (2021) proposed a similar DRL-based JSP solver, utilizing richer node features while explicitly accounting for heterogeneous node relationships during message passing. Zeng et al. (2022) proposed a DRL-based method for FJSP, which represents scheduling states via disjunctive graphs and extracts state features from graphs via GNNs for efficient decision making among agents. Lei et al. (2022) introduced an end-to-end DRL framework that employs disjunctive graphs for state representations and utilizes multi-pointer graph networks (MPGN) to process the structural information of disjunction graphs. They trained operation selection and machine allocation strategies using multiple proximal policy optimization (PPO) algorithms. Zhang et al. (2023a) proposed a two-stage DRL-based method for dynamic FJSP (DFJSP) with variable processing times (FJSP-VPT), utilizing dynamic scheduling experience, representing scheduling states via disjunctive graphs, and applying the GNNs for feature extraction. Lei et al. (2024) proposed a hierarchical DRL

framework based on disjunctive graphs to address DFJSP with unpredictable job arrivals, where an upper-layer agent decomposes DFJSP into multiple FJSPs and a lower-layer agent learns scheduling strategies for FJSPs via GNNs. Yuan et al. (2024) proposed a novel DRL framework to achieve representation learning and policy learning for FJSP, which represented the states by disjunctive graphs and used a lightweight multilayer perceptron (MLP) for state embedding. These studies effectively solve FJSPs by combining GNNs' strength in modeling complex graph structures with DRL's powerful feature perception and decision-making capabilities. Notably, the global states of FJSPs include both operation and machine scheduling states; however, disjunctive graph representations are inherently limited to operation nodes, omitting explicit machine node representations (Munikoti et al., 2024). Consequently, typical GNNs primarily focus on operation node features while ignoring machine node features when processing these graphs. This limitation impacts the training of scheduling strategies to effectively optimize machine utilization and appropriately assign available machines to operations.

As research progresses, research hotspots focus on using heterogeneous graphs to represent state features for FJSPs. Song et al. (2023) proposed a novel scheduling state heterogeneous representation method based on heterogeneous GNN (HGNN) as a state embedding network to describe the complex relationships between operations and machines. Wan et al. (2024) developed an end-to-end DRL framework that combines a meta-path-based HGNN (MHGNN), which enables heterogeneous graphs to manage multiple types of nodes, greatly reducing the graph's density and effectively lowering both computational and storage costs. Nevertheless, whether considering homogeneous or heterogeneous graphs, increasing problem size causes the complexity and density of the graphs to rise rapidly, thus hindering efficient scheduling state feature extraction. Recent advances have highlighted the potential of representation learning techniques, such as GNNs (Wu et al., 2020) and attention mechanisms (Vaswani et al., 2017), in capturing inherent structures of COPs with complex constraints. Chen et al. (2023) presented a novel DRL framework utilizing an attention mechanism and disjunctive graph embedding, which has shown effectiveness in solving large-scale JSP. Yuan et al. (2024) proposed an end-to-end DRL framework for FJSP. They designed a dual attention network (DAN), featuring multiple interconnected attention blocks for operation and machine information, serving as state embedding network to extract feature details. Zhang et al. (2024a) proposed a collaborative agent RL (CARL) framework for FJSP. In CARL, graph attention networks (GATs) are

Table 2

The description of the symbols for FJSP.

Notation	Description
Indices	
i, i'	Index for jobs, i.e., $i, i' = 1, 2, \dots, n$.
j, j'	Index for operations of job, i.e., $j, j' = 1, 2, \dots, n_i$.
k	Index for machines, i.e., $k = 1, 2, \dots, m$.
Constant	
n	Total number of jobs.
n_i	The number of operations belonging to job J_i .
m	Total number of machines.
L	A sufficiently large positive number.
$PT_{ij,k}$	The processing time of O_{ij} on machine M_k .
Sets	
J	Set of jobs, i.e., $J = \{J_1, J_2, \dots, J_n\}$.
M	Set of machines, i.e., $M = \{M_1, M_2, \dots, M_m\}$.
M_{ij}	The optional machine set for operation O_{ij} .
$\mathcal{O}$	Set of all operations.
$\mathcal{O}_i$	Set of operations for job J_i , i.e., $\mathcal{O}_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,n_i}\}$.
$\mathcal{O}'$	The collection of all current operations to be processed.
$\mathcal{N}_k$	Set of adjacent nodes of machine M_k in graph structure.
Variable	
$O_{i,c}$	Currently waiting for scheduling operation of job J_i .
B_{ij}	The start time of operation O_{ij} .
$X_{ij,k}$	$X_{ij,k} = \begin{cases} 1, & \text{if } O_{ij} \text{ selects processing machine } M_k \\ 0, & \text{otherwise.} \end{cases}$
$Y_{ij,j'}$	$Y_{ij,j'} = \begin{cases} 1, & \text{if } O_{ij} \text{ is a predecessor of } O_{i,j'}, i < i' \\ 0, & \text{otherwise.} \end{cases}$

adopted to extract the global states from disjunctive graphs, and a transformer encoder (TE) is used to capture competitive relationships among machines. Wan et al. (2025) proposed a multi-agent-based graph RL (MAGRL) framework to effectively solve FJSP. In MAGRL, state features are extracted via improved GAT (IGAT) based on heterogeneous graph representations. Table 1 summarizes recent research on solving FJSPs by DRL-based methods with diverse graph embedding networks.

Current research trends emphasize enhancing graph structures and employing advanced GNNs to extract meaningful information from complex FJSPs. However, inherent complexity of FJSPs implies that even representing states using heterogeneous graphs encounters significant challenges with large-scale instances, where both graph density and complexity rise sharply. Moreover, existing research has mainly emphasized state feature extraction techniques, with limited investigation of formulating FJSPs as suitable MDPs adapted to the problem characteristics. It is crucial to establish sound MDPs for DRL-based methods, which may affect the efficiency and effectiveness of generating solutions from end-to-end DRL frameworks. Therefore, this study introduces a novel feature extraction method employing heterogeneous graph-based MHSAN to perform multiple feature embeddings, yielding rich and informative representations for FJSPs. This multi-agent DRL framework, based on MHSAN with heterogeneous graph embedding, effectively handles FJSPs across scales while showing state-of-the-art superiority. Furthermore, we investigate the modeling of FJSPs as the MD-MDP, providing promising perspectives for designing rational and efficient methods for modeling MDPs. Building on these, our proposed multi-agent DRL framework (MAMHSAN) exhibits excellent efficacy in solving FJSPs, as detailed and validated in the following sections.

3. Preliminaries and problem formulation

3.1. Description of FJSP

For efficient implementation of FMS, the production process is typically modeled as a FJSP (Xie et al., 2019). The FJSP can be defined as follows. There is a set of jobs $J = \{J_1, J_2, \dots, J_n\}$ processed in the job shop that contains multiple machines $M = \{M_1, M_2, \dots, M_m\}$ and each job J_i consists of n_i operations, i.e., $\mathcal{O}_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,n_i}\}$. These operations must be processed in predefined sequential order, meaning that the

subsequent operation O_{ij} can only commence once the preceding operation $O_{i,j-1}$ has been completed. The flexible production reflects that machine available for processing can be selected and adapted to suit specific situations for different operations. Each operation O_{ij} requires assignment to a processing machine from a set of eligible machines $M_{ij} \in M$, whereas each machine M_k processes at most one operation simultaneously. FJSP involves assigning appropriate processing machines for operations and determining processing sequences of operations on machines. Some assumptions include: 1) All jobs and machines are available at the initial moment; 2) Transport time between operations is negligible; and 3) Once a machine starts processing an operation, it is not allowed to be interrupted. Notation definitions are summarized in Table 2. Following the problem description and definition of notations, the mathematical model is expressed in Eqs. (1)–(7).

$$B_{ij} \geq 0, \forall O_{ij} \in \mathcal{O} \quad (1)$$

$$\sum_{k=1}^m X_{ij,k} = 1, \forall O_{ij} \in \mathcal{O} \quad (2)$$

$$\sum_{k=1}^m \sum_{i=1}^n \sum_{j=1}^{n_i} X_{ij,k} = \sum_{i=1}^n n_i \quad (3)$$

$$B_{ij} + \sum_{M_k \in M_{ij}} PT_{ij,k} X_{ij,k} \leq B_{i,j+1}, \forall O_{ij} \in \mathcal{O} \quad (4)$$

$$B_{ij} + PT_{ij,k} X_{ij,k} \leq B_{i,j'} + L(3 - Y_{ij,j'} - X_{ij,k} - X_{i,j',k}), \quad \forall O_{ij} \in \mathcal{O}, O_{i,j'} \in \mathcal{O}, M_k \in M_{ij} \cap M_{i,j'} \quad (5)$$

$$B_{i,j'} + PT_{i,j',k} \leq B_{ij} + L(2 + Y_{ij,j'} - X_{ij,k} - X_{i,j',k}), \forall O_{ij} \in \mathcal{O}, O_{i,j'} \in \mathcal{O}, M_k \in M_{ij} \cap M_{i,j'} \quad (6)$$

$$C_{\max} \geq B_{i,n_i} + \sum_{M_k \in M_{i,n_i}} (PT_{i,n_i,k} X_{i,n_i,k}), \forall J_i \in J \quad (7)$$

Eq. (1) ensures processing times of all operations must be greater than or equal to the initial time. Eq. (2) indicates that any operation can only be executed on one machine. Eq. (3) indicates that each operation must be assigned to a specific processing machine. Eq. (4) shows that for all operations of the same job J_i , the next operation O_{ij} can be started only after the completion of its preceding operation $O_{i,j-1}$. Eq. (5) and (6) indicates that for different operations executed on the same machine, the subsequent operation O_{ij} can only begin once the previous operation $O_{i,j'}$ has been completed. The goal of FJSP is to reduce the overall completion time for all operations which means minimizing the makespan ($C_{\max}$), with constraints satisfying Eq. (7).

3.2. Heterogeneous graph representation of FJSP

Previous studies commonly utilized disjunction graphs to represent scheduling schemes of FJSPs. However, as the size of problems increases, these graphs may become increasingly complex, resulting in dense graph structures. Thus, extracting meaningful information may pose considerable challenges (Munikoti et al., 2024). To overcome these limitations, researchers have enhanced disjunction graphs using heterogeneous graphs to represent scheduling schemes of FJSPs (Song et al., 2023; Wan et al., 2024, 2025). The heterogeneous graph is defined as a quadruple $\mathcal{G} = (\mathcal{P}, \mathcal{M}, \mathcal{D}, \mathcal{U})$, where $\mathcal{P}$ denotes operation nodes and $\mathcal{M}$ signifies machine nodes. The set $\mathcal{D}$ consists of directed arcs that establish each path connecting virtual start (S) to end (E) nodes, illustrating sequential constraints between operations for each job. The set $\mathcal{U}$ comprises undirected arcs that link operation node ($\mathcal{P}$) to machine node ($\mathcal{M}$), indicating that specific operations can be executed on eligible machines. Heterogeneous graph incorporates two types of nodes (i.e., $\mathcal{P}$ and $\mathcal{M}$), with distinct properties, as well as two types of edges (i.e., $\mathcal{D}$

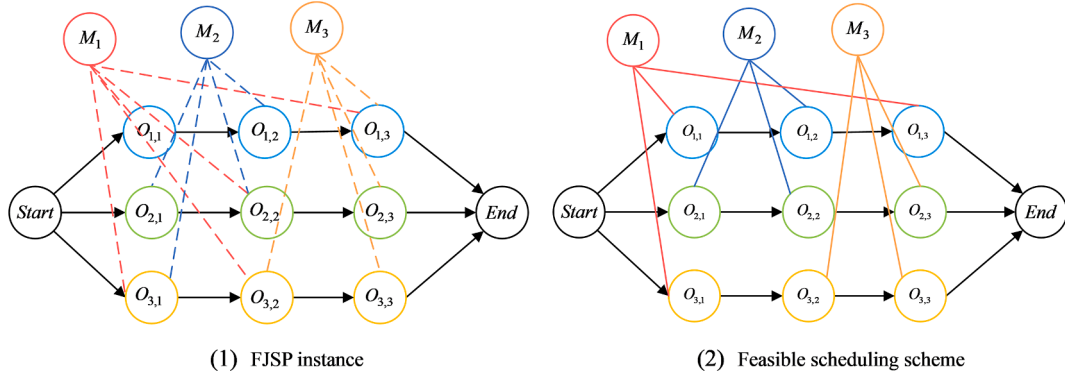


Fig. 1. Heterogeneous graph representation of an instance for FJSP.

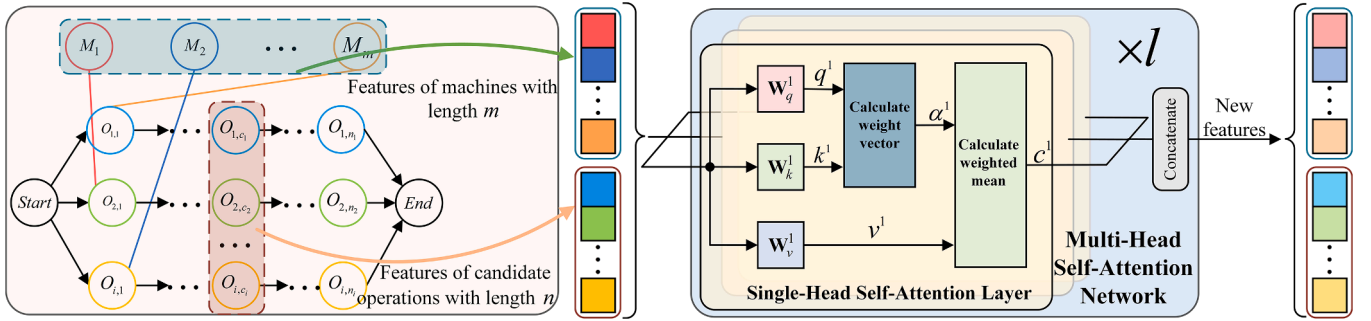


Fig. 2. Feature extraction for FJSP of any scale using MHSAN.

and $\mathcal{H}$), which represents different characteristics, forming a quadruple that forms a complete heterogeneous graph. Fig. 1 illustrates a 3×3 scale instance of FJSP via heterogeneous graph representation. Notably, representing scheduling schemes of FJSPs by the heterogeneous graph effectively adapts multiple node types. By adding undirected arcs $\mathcal{H}$ between corresponding nodes after assigning one operation on a machine, this representation greatly reduces graph density, thereby lowering computational costs and storage costs while enhancing information extraction efficiency (Wu et al., 2021).

3.3. Multi-head self-attention network

The multi-head self-attention network (MHSAN) comprises multiple single-head self-attention networks (SHSAN) (Vaswani et al., 2017), which has inherent strength of being compatible with sequential inputs of arbitrary length without any changes to network architecture. For any layer processing input $X \in \mathbb{R}^{T \times d_{in}}$ (where T denotes sequence length and d_{in} input dimension), we first project it to query (q), key (k), and value (v) vectors via learnable weight matrices $\mathbf{W}_q$, $\mathbf{W}_k$, and $\mathbf{W}_v$ ($\mathbf{W}_q \in \mathbb{R}^{d_{in} \times d_k}$, $\mathbf{W}_k \in \mathbb{R}^{d_{in} \times d_k}$, and $\mathbf{W}_v \in \mathbb{R}^{d_{in} \times d_v}$). Then, based on attention weights derived from q and k , enabling weighted aggregation of v to obtain the outputs, as formalized in Eq. (8). For the l -th layer, the calculation method of outputs is shown in Eq. (9), which depends on the learnable weight parameters of the l -th layer, i.e., $\mathbf{W}_q^l$, $\mathbf{W}_k^l$, and $\mathbf{W}_v^l$. Finally, concatenating the outputs of all layers yields the final results after passing through MHSAN, as shown in Eq. (10).

$$\text{Attention}(q, k, v) = \text{Softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)v \quad (8)$$

$$\text{head}_l = \text{Attention}(X\mathbf{W}_q^l, X\mathbf{W}_k^l, X\mathbf{W}_v^l) \quad (9)$$

$$\text{MHSAN}(X, \mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v) = [\text{head}_1, \text{head}_2, \dots, \text{head}_l] \quad (10)$$

Recent research for FJSPs through DRL-based methods, scholars have introduced the concept of heterogeneous graphs to offer feature-based agents for decisions (Song et al., 2023; Wan et al., 2025). Yet increasing graph density with problem complexity impedes efficient feature extraction via graph neural networks (GNNs), substantially increasing computational and storage cost. Since the scales of datasets are not consistent in scheduling environments, both training and inference in such situations always require holding uniform dimensions, with this limitation decreasing the efficiency of feature extraction for keeping consistent graph topology. As a result, these issues motivate to develop a new framework of HGNNs with attention mechanisms that serves as effective graph-embedded networks (Guo et al., 2022).

The combination of heterogeneous graphs and MHSAN can effectively resolves these challenges. As shown in Fig. 2, for any instance with m machines and n jobs, if MHSAN is employed as graph embedding network, it can perform feature association and aggregation through MHSAN, effectively associating features between machines and jobs. Due to inherent compatibility of MHSAN for inputs with any length, it enables adaptation to diverse scales of FJSPs without architectural modifications. This is very meaningful for DRL-based methods to solve FJSPs, which allow for agent training using multiple instances of different scales fed to networks, capable of extracting multimodal features and promising patterns that make the agents have strong generalization capabilities. Specifically, utilizing MHSAN can effectively eliminate the generalization limitations of traditional graph networks caused by different numbers of jobs and machines for different scales of FJSPs. Moreover, heterogeneous graphs enable the correlation of state features between operations and machines, resulting in richer and multimodal perceptual representations. Hence, the application of graph embedding network using MHSAN for heterogeneous graphs enables efficient feature extraction across diverse instance scales.

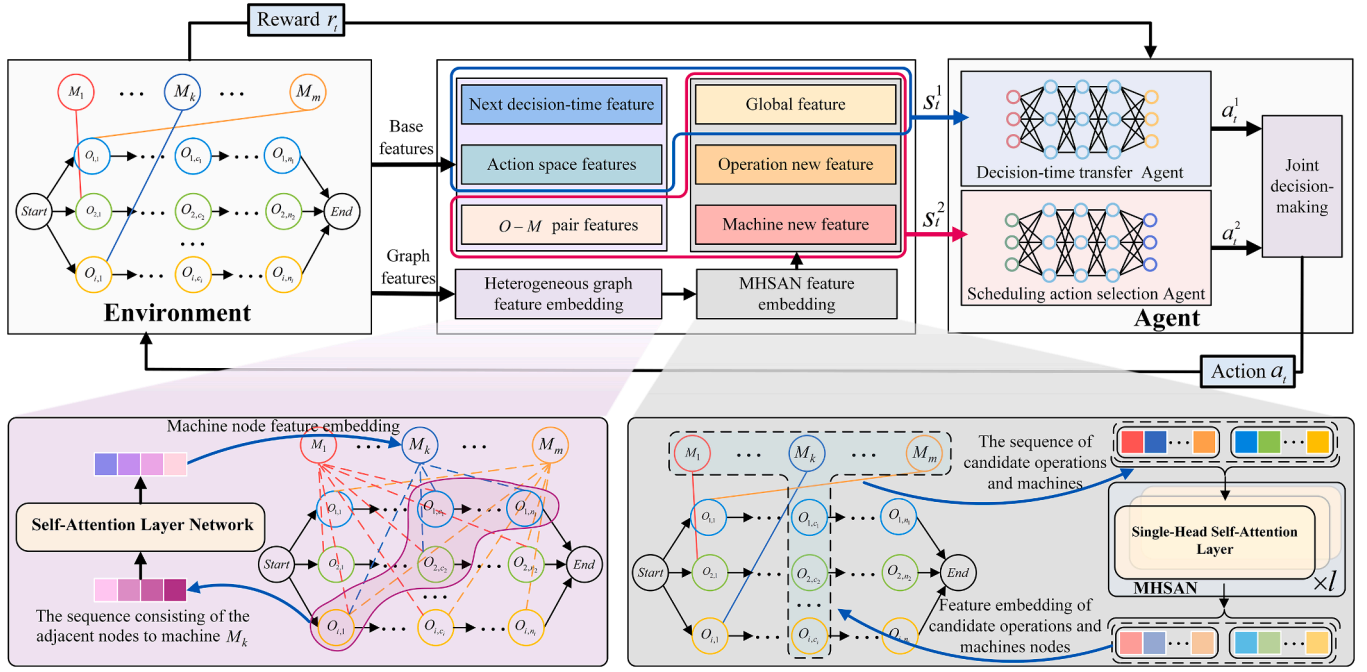


Fig. 3. The overall framework of the MAMHSAN.

4. Proposed method

This section provides a detailed introduction to the multi-agent reinforcement learning framework proposed for solving FJSPs. First, the limitations of traditional MDP modeling methods are analyzed, and then a more advanced approach for modeling FJSP as a new MDP, called MD-MDP, is introduced. Next, a two-stage feature extraction method combining heterogeneous graph feature embedding with MHSAN is developed to address association and aggregation of features for arbitrary scales. Finally, a multi-agent DRL framework based on the proposed MD-MDP is developed, enabling high-quality solutions for FJSPs through joint decision-making among agents. Additionally, a PPO-based training algorithm suitable for this MADRL framework is created. The overall process of the proposed method is illustrated in Fig. 3, with detailed explanations in subsections below.

4.1. Limitations of traditional MDP modeling method in FJSP

The core challenge in using DRL-based methods to solve FJSPs lies in formulating the scheduling decisions as MDPs. For the FJSP, it aims at assigning a feasible processing machine to each operation while determining the sequence of operations to be executed on each machine. For traditional method of modeling FJSPs as MDPs, these processes are implemented through agent decisions from the valid action space $|A_{\text{valid}}|$ at each decision time $T(t)$, where t denotes the decision step. At the initial time ($t = 1, T(1) = 0$) when the agent needs to make the first decision, the action space $|A|$ is composed of all $O-M$ pairs formed by pairwise combinations of each operation O_{ij} and each machine M_k . Note that an operation O_{ij} becomes available when its release time is less than or equal to $T(t)$, while a machine M_k is free when its release time is less than or equal to $T(t)$. It should be noted that not all $O-M$ pairs are executable actions; only when the process O_{ij} corresponding to job J_i is in available state, the corresponding M_k is in idle state, and the operation O_{ij} can be processed on M_k , it can be considered as a valid scheduling action. All valid actions constitute the valid action space $|A_{\text{valid}}|$. At each decision time $T(t)$, the agent selects an action from $|A_{\text{valid}}|$, executes it and transitions to the next state, yielding a new state with updated $|A_{\text{valid}}|$ for the agent to make action decisions for $T(t+1)$. As the envi-

Table 3

An example of 3×3 scale instance for FJSP.

Jobs	Operations	Optional machines and processing time		
		M_1	M_2	M_3
J_1	$O_{1,1}$	3	4	4
	$O_{1,2}$	2	1	1
J_1	$O_{2,1}$	4	4	2
	$O_{2,2}$	1	2	4
J_3	$O_{3,1}$	2	8	2
	$O_{3,2}$	1	4	4

ronment transfers to the next state and $|A_{\text{valid}}| = \{\emptyset\}$, it will transfer $T(t)$ to the next state with $|A_{\text{valid}}| \neq \{\emptyset\}$, referred to as $T'(t)$. This process of repeatedly selecting actions continues until a final solution is found for an instance of FJSP. To describe the proposed modeling method intuitively, we present a 3×3 instance (3 machines, 3 jobs, 3 operations per job) for illustration. The processing time of every operation on each machine is shown in Table 3.

For this FJSP case study, we first use the traditional MDP method to solve it, as shown in Fig. 4. This traditional MDP modeling method has been adopted in existing works (Song et al., 2023; Yuan et al., 2024). In Fig. 4, available jobs and idle machines are marked in green, while illegal and legal $O-M$ pairs are marked in red and black, respectively. All $O-M$ pairs constitute the action space $|A|$. For the determined scheduling actions, they are labeled on Gantt charts and heterogeneous graphs. At each decision step t , the agent needs to make appropriate action choices based on the scheduling state. As shown in Fig. 4 (a), the red dashed line represents the current decision time $T(t)$ at initial moment, with all jobs ($J_1 \sim J_3$) and machines ($M_1 \sim M_3$) release times at 0. Since all operations are processed on any machine, the action space at this time is $|A| = |A_{\text{valid}}|$. Based on this, the agent makes its first scheduling decision at $t = 1$ and $T(1) = 0$. Assuming that the action chosen by the agent from $|A_{\text{valid}}|$ is $O_{2,1} - M_3$ (determined actions are marked by orange dashed box in $|A|$ and thick orange lines in the heterogeneous graph), the environment executes this action to trigger state transition for new decision, as shown in Fig. 4 (b). This MDP for agent decisions driving environment state transitions repeatedly iterates until

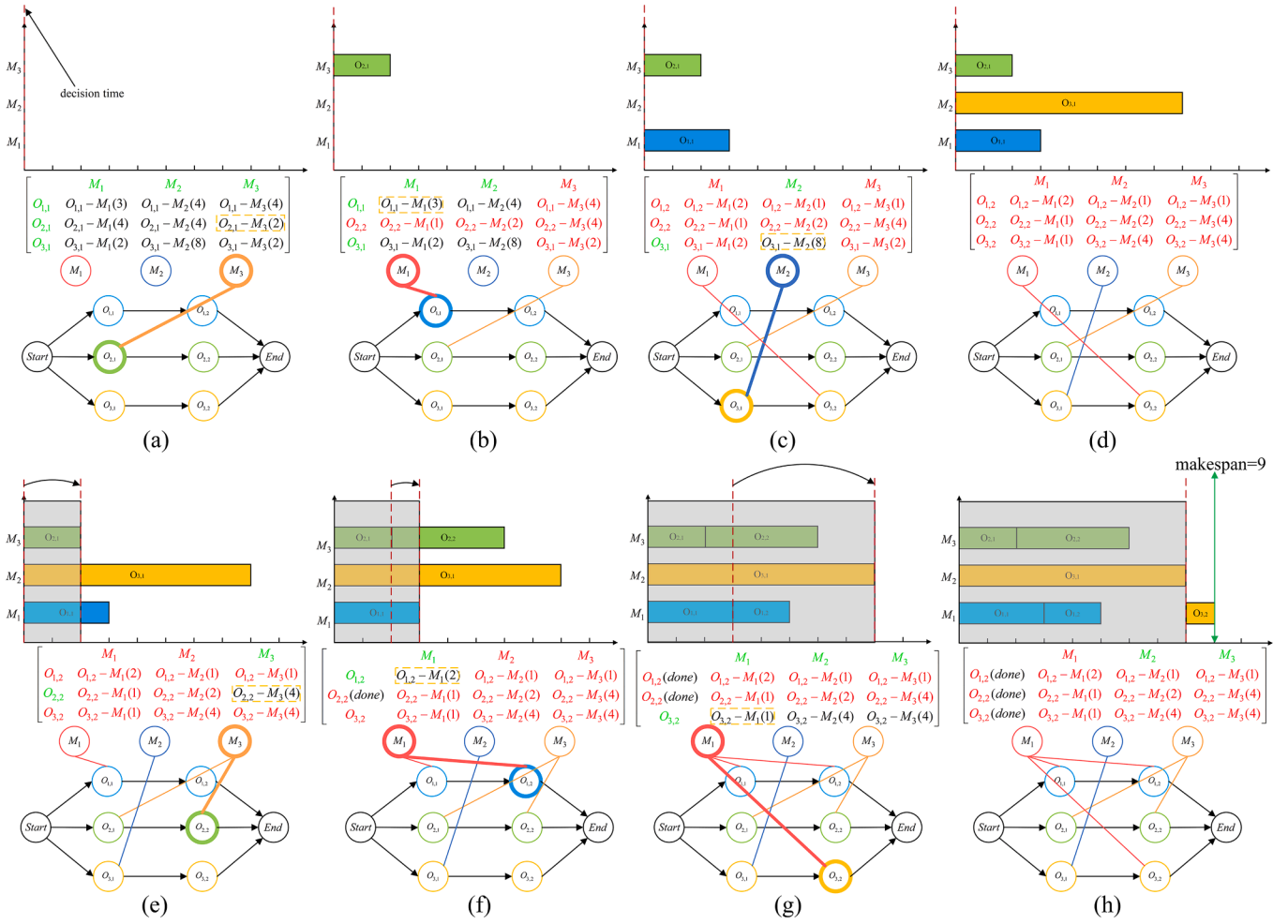
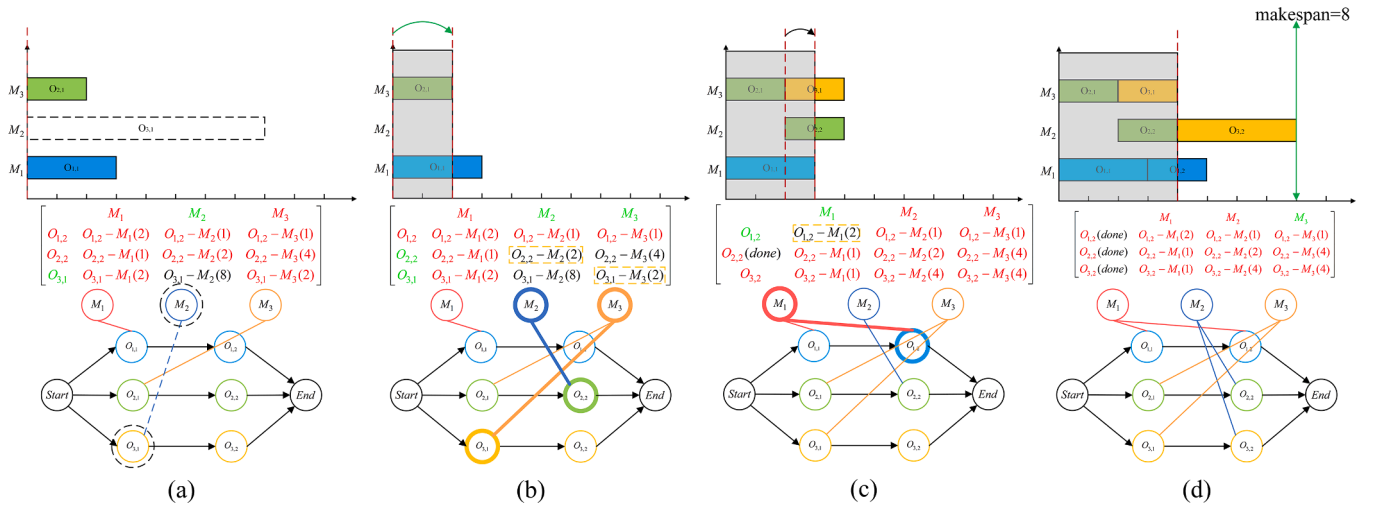


Fig. 4. Scheduling decision process for solving FJSP with traditional MDP.

Fig. 5. Scheduling changes with decision time $T(t)$ transfer in Fig. 4 (c).

a scheduling solution is found. It is intended to point out the limitations of modeling the FJSP as the traditional MDP through the illustrations. Thus, in Fig. 4, the agent's scheduling actions are made randomly from the valid action space $|A_{\text{valid}}|$. For FJSP's instance shown in Table 3, the final scheduling scheme is shown in Fig. 4 (a)-(h), with the makespan of 9 shown in Fig. 4 (h).

From the final scheduling results, it can be easily observed that the action choice $O_{3,1} - M_2$ affects the final decision. As shown in Table 1, the processing times for operation $O_{3,1}$ on M_1 and M_3 are shorter compared to M_2 . If the operation $O_{3,1}$ can be processed on M_1 or M_3 , the final makespan will be reduced, thereby improving the quality of scheduling solution. Let us return to the scheduling decision for

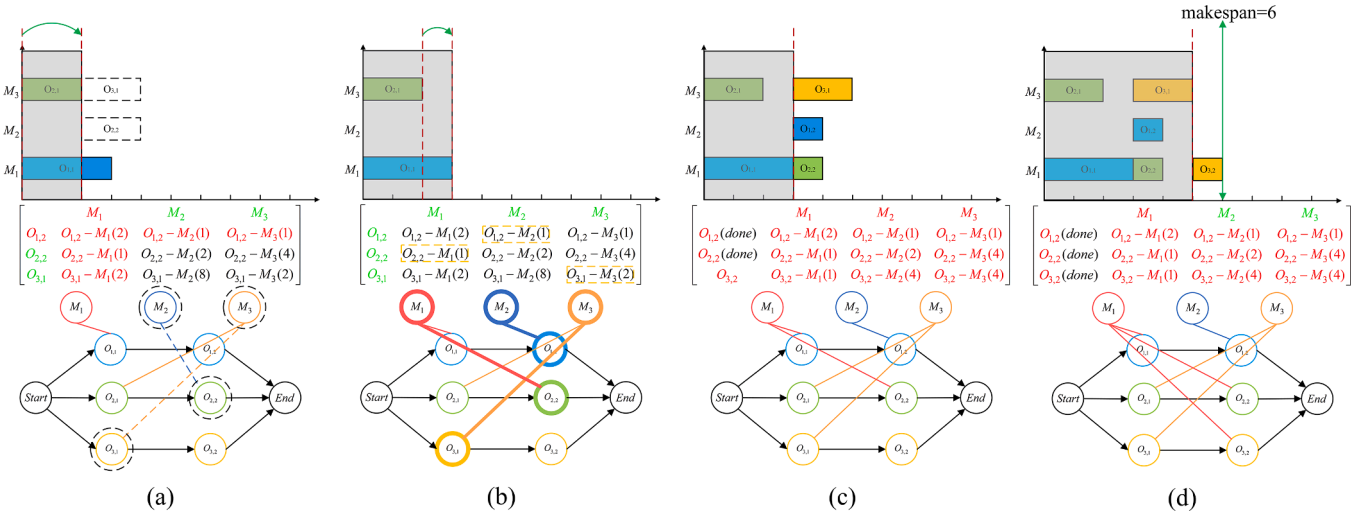


Fig. 6. Scheduling changes with decision time $T(t)$ transfer in Fig. 5 (b).

$O_{3,1} - M_2$, as shown in Fig. 4 (c), at which point the only action left in $|A_{\text{valid}}|$ is $O_{3,1} - M_2$. In traditional MDP modeling, as long as the $|A_{\text{valid}}|$ is not empty, i.e., $|A_{\text{valid}}| \neq \{\emptyset\}$, the agent should select an action from $|A_{\text{valid}}|$. So, in this state, the agent has to choose $O_{3,1} - M_2$ and cannot cancel the scheduled operations. If we want the agent to have more optional actions, that is, to expand the $|A_{\text{valid}}|$, then flexibly transferring decision time $T(t)$ is a wise way.

So, what if we could handle the $T(t)$ more flexibly? Let us return to the situation in Fig. 4 (c): at this decision time (i.e., $T(3) = 0$), the release times of machine M_1 and job J_1 are 3, and those of machine M_3 and job J_2 are 2. As a result, machines M_1 and M_3 are occupied, and jobs J_1 and J_2 are unavailable, leaving only action $O_{3,1} - M_2$ as a valid option in the action space. Following existing methods, the agent can only be forced to choose this action $O_{3,1} - M_2$, as shown in Fig. 5 (a). However, it is clear from the analysis and results in Fig. 4 that the choice of this action will produce suboptimal results. To break this status quo, we attempt to change the current time $T(3) = 0$ by shifting the $T(t)$ to $T'(t)$, i.e., $T(3) \Rightarrow T'(3)$, as shown in Fig. 5 (b). This adjustment makes jobs J_1 and J_3 available, while machines M_2 and M_3 become idle, thus increasing the number of valid actions in the action space. As a result, $O_{3,1}$ no longer has to be processed on M_2 but can choose $O_{3,1} - M_3$ with a shorter processing time; similarly, $O_{2,2}$ is no longer enforced to be processed on M_3 , but it has the option of choosing $O_{2,2} - M_2$ with shorter processing time. Assume that the options of assigning $O_{3,1}$ to M_3 and $O_{2,2}$ to M_2 are chosen, as shown in Fig. 5 (b) and (c). To reflect the decision effect of the overall action choices by altering $T(t)$, existing methods are adopted for scheduling subsequent operations, sequentially assigning $O_{1,2}$ to M_1 and $O_{3,2}$ to M_2 (see Fig. 5 (d)). Compared to Fig. 4, this scheme results in shorter processing times for $O_{2,2}$ and $O_{3,1}$, achieving superior $\text{makespan} = 8$.

The state in Fig. 5 (b) can be derived even further: the release times for machine M_1 and job J_1 are 3, and for machine M_3 and job J_2 are 2 when $T(3) = 2$. At this time, machine M_1 is occupied and job J_1 is unavailable. If existing methods are used, the agent will face two scheduling actions: $O_{3,1} - M_3$ and $O_{2,2} - M_2$ (see Fig. 6 (a)). For the scheduled operation $O_{2,2}$ of job J_2 , its processing time on M_1 is the shortest ($PT_{2,2,1} = 1$), yet machine M_1 is occupied. To break this status quo, we decided to adjust $T(3) = 2$ by shifting it to $T'(t)$, i.e., $T(3) \Rightarrow T'(3)$, which corresponds to the release times of machine M_1 and job J_1 (see Fig. 6 (b)). At this time, jobs J_1 , J_2 , and J_3 are all available, and machines M_1 , M_2 , and M_3 are idle, making all actions in the action space valid. For operation $O_{3,1}$, it has the shortest processing time option of $O_{3,1} - M_3$ ($PT_{3,1,3} = 2$); for operation $O_{2,2}$, the option is $O_{2,2} - M_1$ with the shortest processing time ($PT_{2,2,1} = 1$); and for operation $O_{1,2}$, the shortest

processing time option is $O_{1,2} - M_2$ ($PT_{1,2,2} = 1$). Based on this adjustment, the decision is made to schedule $O_{3,1}$ to M_3 , $O_{2,2}$ to M_1 , and $O_{1,2}$ to M_2 , respectively (see Fig. 6 (c)). Similarly, to reflect the decision effect of all action choices by altering $T(t)$, existing methods are used for scheduling of subsequent operations, resulting in $O_{3,2}$ being assigned to M_1 , as shown in Fig. 6 (d). Compared to the scheduling scheme described in Fig. 4, this scheduling scheme yields optimal processing for $O_{1,2}$, $O_{2,2}$, and $O_{3,1}$, while significantly reducing their final complete times with $\text{makespan} = 6$. It is clear that by appropriately adjusting the decision time $T(t)$ in the scheduling process, scheduling solutions obtained are already significantly better than those achieved by existing methods. Existing modeling methods have one obvious drawback: the agent must make decisions from a non-empty set of valid actions, i.e., $|A_{\text{valid}}| \neq \{\emptyset\}$, regardless of whether or not actions are appropriate. This inappropriate decisions may lead to two issues: Firstly, each operation is forced to choose an unsuitable processing machine, resulting in prolonged processing times. Secondly, incorrect scheduling at critical decision times may result in occupied machines, thus preventing suitable processing machines for subsequent operations. For the first scenario, as shown in Fig. 4 (c), by opting to shift the $T(t)$ from $T(3) = 0$ to $T(3) = 2$ in Fig. 5 (b), it can avoid the processing of operation $O_{3,1}$ on machine M_2 (where $O_{3,1}$ has the longest processing time on machine M_2 with $PT_{3,1,2} = 8$). For the second scenario, as shown in Fig. 6 (b), if no available action is undertaken and $T(3) = 2$ is changed to $T(3) = 3$, effectively waiting for machine M_1 to complete operation $O_{1,1}$, operation $O_{2,2}$ gains the opportunity to be processed on machine M_1 ($O_{2,2}$ has the minimum processing time on machine M_1 with $PT_{2,2,1} = 1$). Accordingly, if the $T(t)$ can be adjusted in time based on the current state of the action space in scheduling process, the agent will have broader and better choices, which is beneficial for scheduling decisions. Based on this insight, we propose a new method for modeling FJSPs as MDPs to address this deficiency.

4.2. Multi-dimensional MDP

Drawing from the preceding discussion, this section first introduces a novel method for modeling FJSPs as effective MDPs. We enhance existing modeling methods by incorporating transfer actions and multiple rewards, namely multi-dimensional action and reward MDP (MD-MDP). This modeling approach effectively overcomes limitations posed by constrained action spaces. In MD-MDP, at the t -th step, the scheduling action a_t is determined jointly by two actions (a_t^1 and a_t^2): a_t^1 recognizes the potential transition from the current decision time $T(t)$ to $T'(t)$, and a_t^2 determines the selection of scheduling actions from the

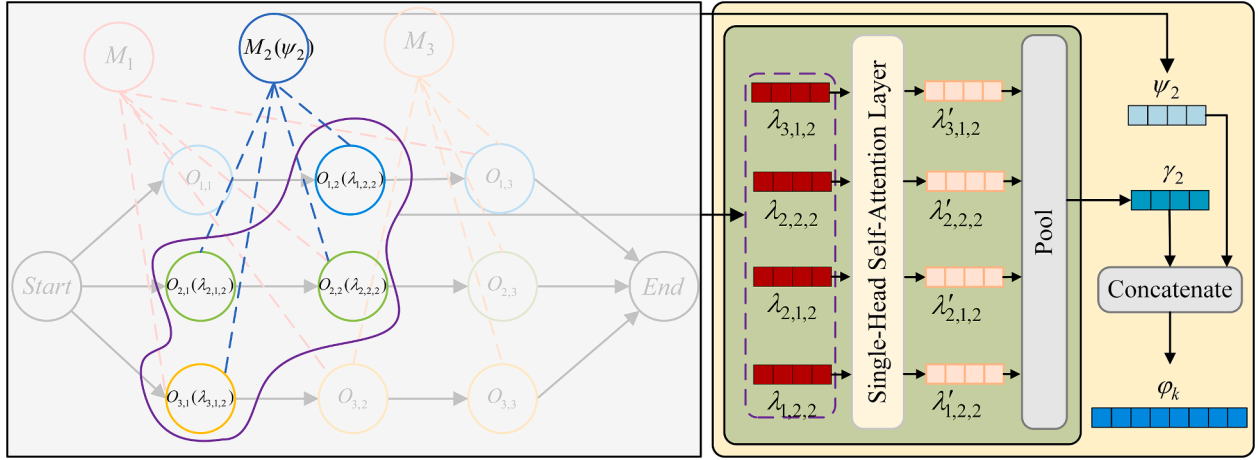


Fig. 7. Schematic of feature extraction for machine nodes in MHSAN.

current action space $|A|$. If the action a_t^1 involves transferring from $T(t)$ to $T'(t)$, then $a_t = a_t^1$; otherwise, $a_t = a_t^2$. In MD-MDP, multiple rewards are formulated based on the effects of joint action a_t . Specifically, if $a_t = a_t^1$, the reward is $r_t = r_t^1$, and if $a_t = a_t^2$, the reward is $r_t = r_t^2$. Detailed reward definitions are provided in subsequent sections. To optimize agent decision-making, the state s_t is decomposed into two components: s_t^1 and s_t^2 . s_t^1 contains complete states in global scheduling environment and the action space $|A|$ at decision time $T(t)$, which enables the agent to refine decision time $T(t)$ for improved decision efficacy. s_t^2 represents both the state of the scheduling environment and the relationship between current operations to be scheduled and the machine at decision time $T(t)$. This informs the agent's selection of superior scheduling actions from the action space $|A|$. The MD-MDP is defined as follows:

State Definition: The state s_t consists of s_t^1 and s_t^2 . s_t^1 represents the state of the current action space, and s_t^2 represents the overall state of the current scheduling environment. The original feature ϕ_k of machine M_k features has three parts: the adjacent node feature $\lambda_{ij,k}$ from the heterogeneous graph representation, the feature $\lambda'_{ij,k}$ derived via SHSAN pooling, and the corresponding machine node in heterogeneous graph concatenated with γ_k . This feature composition is defined by Eq. (11). The global feature is composed of the original operation feature μ_{ij} and machine feature ϕ_k , which are pooled and concatenated to yield new features μ'_{ij} and ϕ'_k through their respective MHSANs, as formalized in Eq. (13). Consequently, the state s_t^1 is composed of the global features ϕ , action space features α_T , and the next decision time feature σ_T concatenated together (see Eq. (14)). Similarly, the state s_t^2 consists of the global features ϕ , the original $O-M$ pair features $\xi_{ij,k}$, and new operation features and machine features ν' (see Eq. (15)). At the decision time $T(t)$, detailed definitions of the original features μ_{ij} , $\xi_{ij,k}$, ψ_k , $\lambda_{ij,k}$, the action space feature α_T and α_T are provided in Appendix.

$$\gamma_k = \frac{1}{|\mathcal{N}_k|} \sum_{O_{ij} \in \mathcal{N}_k} \lambda'_{ij,k} \quad (11)$$

$$\phi_k = [\gamma_k \| \psi_k] \quad (12)$$

$$\phi = \left[\frac{1}{|\mathcal{O}'|} \sum_{O_{ij} \in \mathcal{O}'} \mu'_{ij} \parallel \frac{1}{|M|} \sum_{M_k \in M} \phi'_k \right] \quad (13)$$

$$s_t^1 = [\phi \| \eta_T \| \sigma_T] \quad (14)$$

$$s_t^2 = [\phi \| \mu'_{ij} \| \phi'_k \| \xi_{ij,k}] \quad (15)$$

Joint Action: In MD-MDP, the joint action a_t comprises two distinct

dimensions of actions (a_t^1 and a_t^2). Among them, a_t^1 represents transferring the current decision time from $T(t)$ to $T'(t)$, and a_t^2 represents executing a scheduling action selected from the valid action space $|A_{\text{valid}}|$ composed of feasible $O-M$ pairs, which means immediately processing the process O_{ij} corresponding to a_t^2 on machine M_k .

State Transition: The states of the environment change primarily during the execution of actions and the automatic transition of decision times in the environment. Specifically, the two actions a_t^1 and a_t^2 proposed in this method correspond to different state transition methods for the environment. If at decision time $T(t)$ executes the action $a_t = a_t^1$, the environment modifies $T(t)$ to transition it to $T'(t)$. Conversely, if at decision time $T(t)$ executes the action $a_t = a_t^2$, the action corresponding to a_t^2 is carried out, and the scheduling states of both operation O_{ij} and machine M_k associated with this scheduling action is updated and maintained. After the agent has executed the action a_t^2 , if it is found that the set of the valid action space $|A_{\text{valid}}|$ is empty (i.e., $|A_{\text{valid}}| = \{\emptyset\}$) at next time $T(t+1)$, the environment adaptively modifies $T(t+1)$, transitioning it to the decision time $T'(t+1)$.

Reward Design: Within the proposed MD-MDP framework, the calculation method of rewards r_t varies due to different action dimensions. When $a_t = a_t^1$, the reward r_t is calculated as the time difference between $T'(t)$ and $T(t)$, as defined in Eq. (16). When $a_t = a_t^2$, the reward r_t is derived from the completion time before and after the execution of the action. Concretely, $\tilde{\mathcal{C}}(O_{ij,t})$ is defined as the estimated completion time of operation O_{ij} at $T(t)$. The completion time of O_{ij} is determined as the end time of the preceding operation plus the processing time required. The estimated *makespan* is the maximum under the current environment among all operations' estimated completion times, i.e., $\max_{O_{ij} \in \mathcal{O}} \tilde{\mathcal{C}}(O_{ij,t})$. For any operation O_{ij} , if it has been scheduled,

the completion time is the actual completion time. If it has not been scheduled, its completion time represents the completion time of the previous operation O_{ij-1} plus average processing time across available machines (see Eq. (17)). Note that at the initial time before scheduling has begun, all jobs are available, it means, $\tilde{\mathcal{C}}(O_{i,0,t}) = 0$. Through iterative recursion, we compute estimated completion times for all operations, enabling the calculation of the estimated *makespan*. Therefore, by evaluating the estimated *makespan* before and after executing the action a_t^2 , the reward r_t can be determined via Eq. (18).

$$r_t = T(t) - T'(t) \quad (16)$$

$$\tilde{\mathcal{C}}(O_{ij,t}) = \tilde{\mathcal{C}}(O_{ij-1,t}) + \frac{1}{|M_{ij}|} \sum_{k \in M_{ij}} PT_{ij,k} \quad (17)$$

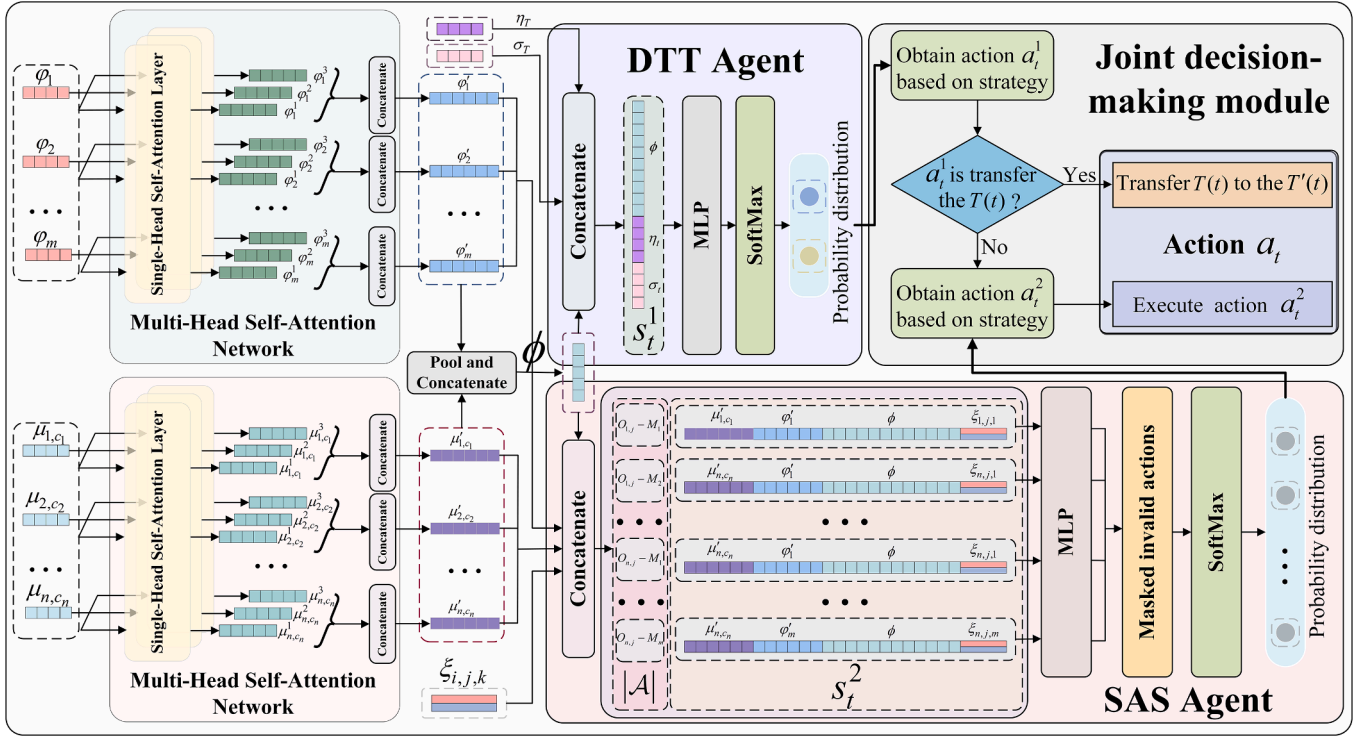


Fig. 8. Schematic of state embedding for multi-agent joint decision-making in MAMHSAN.

$$r_t = \max_{O_{ij} \in \mathcal{O}} \widetilde{\mathcal{E}}(O_{ij,t}) - \max_{O_{ij} \in \mathcal{O}} \widetilde{\mathcal{E}}(O_{ij,t+1}) \quad (18)$$

4.3. Heterogeneous graph-based MHSAN framework

To effectively address FJSPs with various scales and handle both storage and computational costs of feature extraction based on graph structures, this section proposes a new feature extraction method for the scheduling environment of FJSPs through MHSAN with heterogeneous graphs. In this method, the first feature embedding is performed on each machine node and its adjacent operations nodes via an attention network based on the representation of scheduling of FJSPs using heterogeneous graphs, as shown in Fig. 7. Due to the fact that the set M_{ij} of selectable machines for each operation O_{ij} is never empty, the feature embedding for each machine effectively utilizes graph structures for feature representation, effectively capturing the potential connections of complex relationships between the operations and machines for FJSPs. Based on this, machine features embedded after the first feature embedding and the original features of the carefully designed operations are passed through MHSAN for the second feature embedding, as shown in Fig. 8. This process inputs all machine and operation features as sequences with length m and n into their respective MHSANs for mapping new features. Depending on these advantages of MHSAN, MHSAN can effectively process any number of machine and operation features, while making the feature information between each machine and operation interrelated, effectively extracting the overall feature information from scheduling environments for agents to make high-quality decisions.

4.3.1. Machine feature extraction based on heterogeneous graphs

In the MHSAN formulated by heterogeneous graph representation, we have designed as follows: each machine node contains its feature, denoted as ψ_k , while its adjacent nodes (all operation nodes) form the set $\mathcal{N}_k$. Each operation node in $\mathcal{N}_k$ contains features represented as $\lambda_{ij,k}$. The SHSAN is employed to extract features from these operation nodes in $\mathcal{N}_k$. The features of all operation nodes in $\mathcal{N}_k$ are arranged into a sequence, then transformed by SHSAN through node-wise correlations

to generate new features $\lambda'_{ij,k}$. Subsequently, the average pooling is used to these new features of these nodes yielding the aggregated feature γ_k (see Eq. (11)). This feature ψ_k is concatenated with γ_k of the machine node M_k to yield final feature ϕ_k of machine M_k (see Eq. (12)). The details of SHSAN, implemented through trainable weight matrices $\mathbf{W}^1$, $\mathbf{W}^2$, and $\mathbf{W}^3$, is formalized in Eqs. (19)–(20). By extracting features from adjacent nodes across all machine nodes, the features of machine nodes can effectively reflect the scheduling states. Fig. 7 illustrates this process of extracting the features of machine nodes.

$$\alpha_{ij,k} = \text{SoftMax}_{O_{ij} \in \mathcal{N}_k} (\langle \mathbf{W}^1 \lambda_{ij,k}, \mathbf{W}^2 \lambda_{ij,k} \rangle) \quad (19)$$

$$= \frac{\exp(\langle \mathbf{W}^1 \lambda_{ij,k}, \mathbf{W}^2 \lambda_{ij,k} \rangle)}{\sum_{O_{ij} \in \mathcal{N}_k} \exp(\langle \mathbf{W}^1 \lambda_{ij,k}, \mathbf{W}^2 \lambda_{ij,k} \rangle)}$$

$$\lambda'_{ij,k} = \text{LeakyRELU} \left(\sum_{h=1}^{|\mathcal{N}_k|} \alpha_{ij,k,h} \mathbf{W}^3 \lambda_{ij,k,h} \right), \forall O_{ij} \in \mathcal{N}_k \quad (20)$$

4.3.2. Machine and operation features embedding

Through feature embedding based on heterogeneous graphs, each node of machine M_k contains the features of its adjacent node set $\mathcal{N}_k$. However, this method does not capture the information of operations nodes that cannot be processed on machine M_k . Meanwhile, the potential relationships between various operations in the environment are equally important for decision-making of agents. To address these limitations, we apply MHSAN to perform secondary feature embedding on machine features after the first feature embedding and original features of the designed process, as shown in Fig. 8. Specifically, in their respective MHSANs, the calculation of attention coefficients $\alpha_{i,c_l,i}^l$ and $\alpha_{k,k'}^l$ between machines M_k and $M_{k'}$ in layer l , as well as between operations O_{i,c_l} and $O_{i',c_l'}$, is shown in Eq. (21). These coefficients yield the output results μ_{i,c_l}^l , ϕ_k^l of machine features ϕ_k and operation features μ_{i,c_l} at layer l are defined in Eq. (22). Finally, MHSAN concatenates layer output results to generate new features ϕ' , μ'_{i,c_l} for each machine M_k and

operation O_{i,c_l} , as defined in Eq. (23). Here, $Z_l^1, Z_l^2, Z_l^3, X_l^1, X_l^2$, and X_l^3 denote distinct learnable parameter matrices at layer l .

$$\begin{cases} \alpha_{k,k}^l = \frac{\exp(\langle Z_l^1 \varphi_k, Z_l^2 \varphi_k \rangle)}{\sum_{M_k \in M} \exp(\langle Z_l^1 \varphi_k, Z_l^2 \varphi_k \rangle)} \\ \alpha_{i,c_l,i}^l = \frac{\exp(\langle X_l^1 \mu_{i,c_l}, X_l^2 \mu_{i,c_l} \rangle)}{\sum_{J_l \in J} \exp(\langle X_l^1 \mu_{i,c_l}, X_l^2 \mu_{i,c_l} \rangle)} \end{cases} \quad (21)$$

$$\begin{cases} \varphi_k^l = \text{LeakyRELU} \left(\sum_{M_k \in M} \alpha_{k,k}^l Z_l^3 \varphi_k \right) \\ \mu_{i,c_l}^l = \text{LeakyRELU} \left(\sum_{J_l \in J} \alpha_{i,c_l,i}^l X_l^3 \mu_{i,c_l} \right) \end{cases} \quad (22)$$

$$\begin{cases} \varphi_k' = [\varphi_k^1 \parallel \varphi_k^2 \parallel \dots \parallel \varphi_k^l] \\ \mu_{i,c_l}' = [\mu_{i,c_l}^1 \parallel \mu_{i,c_l}^2 \parallel \dots \parallel \mu_{i,c_l}^l] \end{cases} \quad (23)$$

4.4. Joint decision making by multiple agents

Based on the MD-MDP, this section proposes a multi-agent DRL framework (MAMHSAN) based on MHSAN and heterogeneous graph embedding method for solving the FJSP. As illustrated in Fig. 8, the MHSAN first extracts the designed state features s_t^1 and s_t^2 from the scheduling environment. Based on these states, the two agents collaboratively determine the action to be executed. Following this, agents execute these actions, resulting in a state transition. By iteratively repeating this process, we can achieve scheduling for the FJSP and obtain the final solution. Specifically, the design of two agents to make action decisions based on the states is as follows.

4.4.1. Decision-time transfer agent

The design of the DDT agent is based on the actor-critic network framework. For the DDT agent, the actor-network is the MLP with parameters θ . It receives the input state s_t^1 and outputs probability values over the two actions in the action space $|\Omega|$. The two actions are as follows: Action 1 transit the decision time $T(t)$ from the environment to $T'(t)$, and Action 2 maintain $T(t)$ without change. As defined in Eq. (24), DDT agent's actor network's final layer uses the SoftMax activation function to compute the probabilities for these two actions. Thus, the DDT agent's actor-network derives the probability distribution over actions in the action space $|\Omega|$.

$$\pi_\theta(a_t^1 | s_t^1) = \text{SoftMax}(\text{MLP}_\theta[\phi \| \eta_T \| \sigma_T]) \quad (24)$$

The DDT agent's critic network with parameters ζ is used to evaluate the current state. It takes s_t^1 as input and outputs a scalar value $v_\zeta(s_t^1)$, which represents the state's estimated values. Note that the primary purpose of the critic network is to facilitate the training of the DDT agent's actor network.

4.4.2. Scheduling action selection agent

The design of the scheduling action selection (SAS) agent is also based on the actor-critic network framework. For the SAS agent, the actor-network with parameter ω utilizes the state s_t^2 to generate the probability distribution over the action space $|A|$. The action space $|A|$ comprises multiple $O-M$ pairs formed by the combination of all pending processing orders and all machines under the decision time $T(t)$. Specifically, the SAS agent's actor network first scores the action space $|A|$, as detailed in Eq. (25). After filtering out all infeasible $O-M$ pairs (scores for infeasible $O-M$ pairs are set to negative infinity), a

probability distribution for the scheduling actions is derived based on the scores, as shown in Eq. (26).

$$c(a_t^2 | s_t^2) = \text{MLP}_\omega [\phi \| \mu_{i,j}' \| \varphi_k' \| \xi_{i,j,k}] \quad (25)$$

$$\pi_\omega(a_t^2 | s_t^2) = \frac{\exp(c(a_t^2 | s_t^2))}{\sum_{b_t^2 \in |A|} \exp(c(b_t^2 | s_t^2))} \quad (26)$$

4.5. Training details

The training process of MADRL frameworks requires coordinating agents to optimize policies for environmental adaptation during dynamic interactions, and necessitates designing appropriate reward mechanisms and update rules to achieve cooperative optimization. We employ a randomly generated dataset ε_{tr} with specified scale $(n \times m)$ as the training set to train the DDT and SAS agents over N episodes. The DDT agent comprises a training actor-network π_θ , a behavior actor-network $\pi_{\theta^{old}}$, and a critic-network v_ζ , with respective trainable parameters θ , θ^{old} , and ζ . Similarly, the SAS-agent incorporates a training actor-network π_ω , a behavior actor-network $\pi_{\omega^{old}}$, and a critic-network v_ζ , with trainable network parameters ω , ω^{old} , and ζ . At each training episode's outset, we randomly sample an instance ε_{tr_n} and initialize the environment. The DDT and SAS agents then interact with this environment through the following process: At timestep t , the environment provides the current state; Both agents collaboratively select a joint action a_t ($a_t = [a_t^1, a_t^2]$); The environment executes scheduling action a_t , yielding reward r_t , and transitioning to state s_{t+1} . This interaction persists for T times until all operations are scheduled to appropriate machines. During the interaction, we collect experiences (s_t, a_t, r_t, s_{t+1}) in a replay buffer, where a_t includes a_t^1 provided by the DDT agent and a_t^2 provided by the SAS agent, and r_t is determined by the executed actions. These experiences are employed to update both agents' network parameters. The training procedure is given in Algorithm 1.

$$L_{CLIP}(\theta)_t = \min\{\delta_t^o A_t^1, \text{clip}(\delta_t^o, 1 - \varepsilon, 1 + \varepsilon) A_t^1\} \quad (27)$$

$$L_{MSE}(\zeta)_t = \text{MSE}(U, v_\zeta(s_t^1)) \quad (28)$$

$$L_{CLIP}(\omega)_t = \min\{\delta_t^o A_t^2, \text{clip}(\delta_t^o, 1 - \varepsilon, 1 + \varepsilon) A_t^2\} \quad (29)$$

$$L_{MSE}(\zeta)_t = \text{MSE}(U, v_\zeta(s_t^2)) \quad (30)$$

To optimize both DDT and SAS agent networks, we implement an improved multi-agent proximal policy optimization (MA-PPO) algorithm. Specifically, at each timestep t , the DDT agent and SAS agent compute their respective advantages (A_t^1 and A_t^2) and probability ratios (δ_t^o and δ_t^w) using experience tuple (s_t, a_t, r_t, s_{t+1}) . These values determine the actor-network and critic-network losses per timestep for both agents, with specific calculations derived following Eqs. (27)-(30). When action a_t^1 triggers transitions to decision time $T'(t)$, the SAS agent does not contribute to decision-making, thus its loss is nullified. After aggregating losses across all timesteps, we compute their mean, apply corresponding loss coefficients, and derive the final losses for each agent's actor and critic networks. These composite losses drive network parameter updates via stochastic gradient descent. Finally, the updated network parameters θ and ω then synchronize behavior policy parameters θ^{old} and ω^{old} , repeating this process K times. The complete update procedure is depicted in Algorithm 2.

Algorithm 1 The training process of MAMSAN.

Inputs: DTT-agent's parameters $\{\theta, \zeta\}$, behavior actor-network π_{θ}^{old} ; SAS agent's parameters $\{\omega, \varsigma\}$, behavior actor-network π_{ω}^{old} ; pre-sampled dataset $\mathcal{E}_{tr}$ and validation dataset $\mathcal{E}_{val}$.

```

1: for  $n=1,2,\dots,N$  do
2:   Sample  $\mathcal{E}_{tr_n}$  in  $\mathcal{E}_{tr}$ .
3:   //Based on instance  $\mathcal{E}_{tr}$ , start the  $n$ -th training session
4:   for  $t=0,1,\dots,T$  do
5:     Get  $a_t^1$  using  $\pi_{\theta}^{old}(s_t^1)$ .
6:     if  $a_t^1$  is transferring  $T(t)$  to  $T'(t)$  then
7:       Receive the reward  $r_t$  by Eq. (16) and transit to the  $T'(t)$ .
8:       Collect experience  $(s_t^1, a_t^1, r_t, s_{t+1}^1)$ .
9:       Update state  $s_t \leftarrow s_{t+1}^1$ .
10:      Continue
11:    end if
12:    Get  $a_t^2$  using  $\pi_{\omega}^{old}(\cdot | s_t^2)$ .
13:    Receive the reward  $r_t$  by Eq. (18) and do action  $a_t^2$ .
14:    Collect experience  $(s_t^2, a_t^2, r_t, s_{t+1}^2)$ .
15:    Update state  $s_t \leftarrow s_{t+1}^2$ .
16:  end for
17:  for  $k=1,2,\dots,K$  do
18:    //Optimize the framework  $K$  times using experience
19:    Update the network parameters  $\theta$ ,  $\zeta$ ,  $\omega$ , and  $\varsigma$ , based on the collected experiences, via Algorithm 2.
20:  end for
21:  if ever  $V$  episodes then
22:    //Verify every  $V$  times on the validation set
23:    Validate  $\pi_{\theta}$  on  $\mathcal{E}_{val}$ .
24:  end if
25: end for

```

Outputs: Network parameters θ and ω obtained after training.

After the MADRL framework has been trained for a certain number of iterations, we employ a fixed validation set $\mathcal{E}_{val}$ to evaluate the model's performance. Throughout the experimental design, we have implemented two distinct action selection strategies. The first is greedy selection: we select the action with the highest probability from the probability distributions provided by the DDT agent and the SAS agent for execution. This greedy selection method is utilized for the purpose of validation. The second strategy is stochastic sampling: we sample random actions from the distributions offered by DDT and SAS agents for execution. This stochastic sampling method is applied during the

training phase to encourage the agents to explore the action space as thoroughly as possible. These strategies are crucial for balancing the trade-offs between exploration and exploitation in the proposed MADRL framework. The greedy selection ensures that agents perform optimally using current experience and problem-specific knowledge, while the sampling strategy facilitates finding potentially promising actions that may not be immediately apparent. This dual-strategy framework supports comprehensive evaluation and continuous improvement of the agents' decision-making capabilities.

Algorithm 2: The MA-PPO for MAMHSAN.

Inputs: DTT-agent's parameters $\{\theta, \zeta\}$, behavior actor-network $\pi_{\theta^{old}}$; SAS-agent's parameters $\{\omega, \varsigma\}$, behavior actor-network $\pi_{\omega^{old}}$; information collected from each step t .

- 1: //Represent the number of SAS agent decisions
- 2: Set $h = 0$.
- 3: **for** $t = T, \dots, 1, 0$ **do**
- 4: Calculate the discount return $U_t = \sum_{t'=t}^T U_{t'} \gamma^{t'-t}$;
- 5: Calculate advantages $A_t^1 = U_t - v_{\zeta}(s_t^1)$, $A_t^2 = U_t - v_{\varsigma}(s_t^2)$;
- 6: Calculate probability ratio $\delta_t^{\theta} = \frac{\pi_{\theta}(a_t^1 | s_t^1)}{\pi_{\theta^{old}}(a_t^1 | s_t^1)}$, $\delta_t^{\omega} = \frac{\pi_{\omega}(a_t^2 | s_t^2)}{\pi_{\omega^{old}}(a_t^2 | s_t^2)}$;
- 7: Calculate actor loss $L_{CLIP}(\theta)_t$ of DTT in step t by Eq. (27)
- 8: Calculate critic loss $L_{MSE}(\zeta)_t$ of DTT in step t by Eq. (28).
- 9: **if** a_t^1 is to the $T'(t)$ **then**
- 10: Set actor loss of SAS in step t : $L_{CLIP}(\omega)_t = 0$;
- 11: Set critic loss of SAS in step t : $L_{MSE}(\varsigma)_t = 0$.
- 12: **else**
- 13: Calculate actor loss $L_{CLIP}(\omega)_t$ of SAS in step t by Eq. (29);
- 14: Calculate critic loss $L_{MSE}(\varsigma)_t$ of SAS in step t by Eq. (30).
- 15: Set $h = h + 1$.
- 16: **end if**
- 17: **end for**
- 18: Calculate $L_{CLIP}(\theta)$ for SAS: $L_{CLIP}(\theta) = (-c_{actor}) \times (\sum_{t=1}^T L_{CLIP}(\theta)_t) / T$;
- 19: Calculate $L_{MSE}(\zeta)$ for SAS: $L_{MSE}(\zeta) = c_{critic} \times (\sum_{t=1}^T L_{MSE}(\zeta)_t) / T$;
- 20: Calculate $L_{CLIP}(\omega)$ for DDT: $L_{CLIP}(\omega) = (-c_{actor}) \times (\sum_{t=1}^T L_{CLIP}(\omega)_t) / (T - h)$;
- 21: Calculate $L_{MSE}(\varsigma)$ for DDT: $L_{MSE}(\varsigma) = c_{critic} \times (\sum_{t=1}^T L_{MSE}(\varsigma)_t) / (T - h)$.
- 22: Update parameters $\theta, \zeta, \omega, \varsigma$ according to the calculation $L_{CLIP}(\theta), L_{MSE}(\zeta), L_{CLIP}(\omega), L_{MSE}(\varsigma)$ by using gradient descent algorithm.
- 23: Update behavior actor network parameters $\theta^{old} \leftarrow \theta, \omega^{old} \leftarrow \omega$.

Outputs: $\pi_{\theta}, \pi_{\theta^{old}}, v_{\zeta}, \pi_{\omega}, \pi_{\omega^{old}}, v_{\varsigma}$.

5. Experimental details

This section presents a numerical comparison of MAMHSAN against multiple baseline methods, including: well-known priority dispatching rules (PDRs), effective exact method provided by Google OR-Tools, a conventional genetic algorithm, and a DRL-based method. To comprehensively evaluate its effectiveness in terms of scheduling performance, generalization capability, and computational efficiency, we employ both synthetic instances and established FJSP benchmarks.

5.1. Experimental settings

5.1.1. Configuration

The hardware utilized in this research comprises a 12th Gen Intel (R) Core (TM) i5-12400F CPU. The graphical processing unit featured is the NVIDIA GEFORCE RTX 4060, which has a memory capacity of 16055

MB. The software environment includes Python version 3.8.0, alongside PyTorch, which is present in version 2.1.0. In this study, we conduct a numerical comparison of MAMHSAN against various baseline methods, including well-known PDRs, the exact method provided by Google OR-Tools, a typical genetic algorithm IJA, regular GA (RegGA) (Rooyani & Defersha, 2019), two-stage genetic algorithm (2SGA), and self-learning GA (SLGA) (Chen et al., 2020), and a DRL-based method (Lei et al., 2022; Song et al., 2023; Yuan et al., 2024). To evaluate its effectiveness in terms of scheduling performance, generalization ability, and computational efficiency, we utilize both synthetically generated instances and commonly employed public FJSP benchmarks. Table 4 presents the hyperparameter configurations for the MA-PPO algorithm.

5.1.2. Datasets

In this study, the datasets are categorized into three separate groups: the training set, the validation set, and the testing set. We employ syn-

Table 4
Hyperparameter details.

Hyperparameters	Values
Number of episodes (N)	500
Number of policy updates (K)	3
Number of interval episodes per validation (V)	5
Learning rate (lr)	1×10^{-3}
Policy loss coefficient (c_{actor})	1
Critic loss coefficient (c_{critic})	0.5
Reward discount factor (γ)	0.98
Clipping coefficient (ϵ)	0.2

Table 5
Details of synthetic datasets.

Size ($n \times m$)	n_i	M_{ij}	$PT_{i,j,k}$
10×5	U(5,6)	U(2,5)	U(3,10)
20×5	U(5,6)	U(2,5)	U(3,10)
15×10	U(8,10)	U(4,10)	U(8,20)
20×10	U(8,10)	U(4,10)	U(8,20)

thesis to generate the training set and validation set data to evaluate the effectiveness of the MAMHSAN in the DRL model training. Additionally, to evaluate the effect of different dataset sizes on model training, the training and validation sets are divided into various sizes, specifically 300, 400, 500, and 600. As for the testing set, its purpose is to benchmark the performance of the MAMHSAN against existing algorithms. Therefore, we utilize public datasets, namely Barnes, Brandimarte, Dauzere, and Hurink (edata, rdata, vdata), which can be accessed at <http://people.idsia.ch/~monaldo/fjsp.html>. Table 5 presents a detailed design of the synthetic datasets used in this study. For an FJSP instance with a scale of 10×5 , we set the range of the number of operations for each job to 5 to 6, and for any operation, we set the number of machines that can be processed to a range of 2 to 5, set all operations processing time on the machine to range from 3 to 10. For the scale of 20×5 , we set the range of the number of operations for each job to 5 to 6, and for any operation, we set the number of machines that can be processed to a range of 2 to 5, set all operations processing time on the machine to range from 3 to 10. For the scale of 15×10 , we set the range of the number of operations for each job to 8 to 10, and for any operation, we set the number of machines that can be processed to a range of 4 to 10, set all operations processing time on the machine to range from 8 to 20. For the scale of 20×10 , we set the range of the number of operations for each job to 8 to 10, and for any operation, we set the number of machines that can be processed to a range of 4 to 10, set all operations processing time on the machine to range from 8 to 20.

5.1.3. Evaluation metrics and baselines

For solving FJSP, the objective is to optimize the *makespan*, which represents the total completion time of all jobs. To highlight the performance differences among various algorithms on the dataset, we employ the relative percentage deviation (RPD) metric to assess the performance of algorithms in optimizing the makespan (Pan et al., 2023). The RPD is calculated as shown in Eq. (31), where $C_{\max}^*$ is the makespan obtained by a specific algorithm for each instance, and $C_{\max}^*$ is the solution obtained by Google OR-Tools, which can be accessed at <https://developers.google.cn/optimization>. For each FJSP instance, we use Google OR Tools to run for 30 min and obtain the result $C_{\max}^*$.

$$RPD = \frac{C_{\max} - C_{\max}^*}{C_{\max}^*} \times 100\% \quad (31)$$

To evaluate the effectiveness of each algorithm involved in this study, a series of comparative experiments with ten different algorithms were carried out. For these metaheuristic methods, including the IJA,

RegGA (Rooyani & Defersha, 2019), 2SGA, and SLGA (Chen et al., 2020), the data for these algorithms mainly comes from the data provided in their respective papers. As for composite PDRs, we utilize the FIFO, MWKR, and MOPNR methods, which have demonstrated robust performance (Sels et al., 2012). It is worth mentioning that we have introduced EET (i.e., early end time) based on these rules to form composite rules for solving FJSP. These composite rules have shown good performance in experiments conducted in multiple studies (Lei et al., 2024; Wan et al., 2024; Yuan et al., 2024). Meanwhile, in order to fully verify the solving performance and generalization ability of MAMHSAN, we compared it with three state-of-the-art DRL based methods. These three are the DRL models proposed by Song et al. (2023) based on heterogeneous graph neural network feature extraction, Lei et al. (2022) based on discrete graph feature embedding, and Yuan et al. (2024) based on lightweight MLP feature extraction. These three DRL-based models for solving FJSP represent the most popular feature extraction methods currently available, and their reproducibility comes from the source code provided in their respective papers. Source code details are available at https://github.com/HuangTongShuaiGe/MAMHSAN_for_fjsp.

5.2. Experimental results

5.2.1. Analysis of policy training

To verify the effectiveness of the proposed MA-PPO algorithm for MAMHSAN, we tested the MAMHSAN trained on multiple training sets of different sizes. Specifically we configured the training datasets with scales of 10×5 , 20×5 , 15×10 , and 20×10 , each consisting of 100 instances, with a total of 1000 iterations conducted, and validated the MAMHSAN every 5 iterations during the training process. The validation datasets matched the scales used in the training, also comprising 100 instances. For each validation, the average makespan of all instances in the validation set was adopted as the performance metric for the MAMHSAN on the validation set. As shown in Fig. 9 the MAMHSAN demonstrates good convergence across training datasets of varying scales, reflecting the rationality of the proposed algorithm's framework design and the feasibility of the proposed MA-PPO.

To verify the generalization ability of the MAMHSAN, we tested it on common examples based on models trained on different training sets. For each FJSP instance in the dataset, we will use Google OR Tools to solve them separately, and the specific detailed data will be provided in Tables 8–13. Based on the results of Google OR Tools, use it as the upper bound for solving the FJSP instance, and calculate its gap through Eq. (31) to demonstrate the quality of the algorithm's solution. To facilitate a clearer and more intuitive display of the MAMHSAN's generalization ability, the performance of the MAMHSAN on a certain dataset for each training set is represented by taking the average gap of all FJSP instances on that dataset. The specific experimental results are shown in Table 6. From Table 6, it can be seen that due to different training sets, the performance of MAMHSAN on the same dataset varies. Specifically, when using a training set with a size of 10×5 , MAMHSAN performed the best on all datasets, even showing an average gap of 1.63 % on the Hurink (vdata) dataset. This result proves that MAMHSAN's solving performance is very close to Google OR Tools to a certain extent. When using training sets of 15×10 and 20×10 scales, their solving performance is somewhat similar. Compared to other training sets, when using a training set with a size of 20×5 , the solving performance of MAMHSAN is relatively low. To some extent, due to the inconsistent problem size between the training and testing sets, the model may not have good solving performance. Currently, the generalization mechanism for DRL model networks is still in its infancy (Witty et al., 2021). Meanwhile, in order to verify whether the proposed MAMHSAN can efficiently face FJSP of various scales, we recorded its average solution time on each dataset in Table 6. The method for calculating the average solution time is to divide the total solution times of each instance in MAMHSAN within each dataset by the number of FJSP instances in that

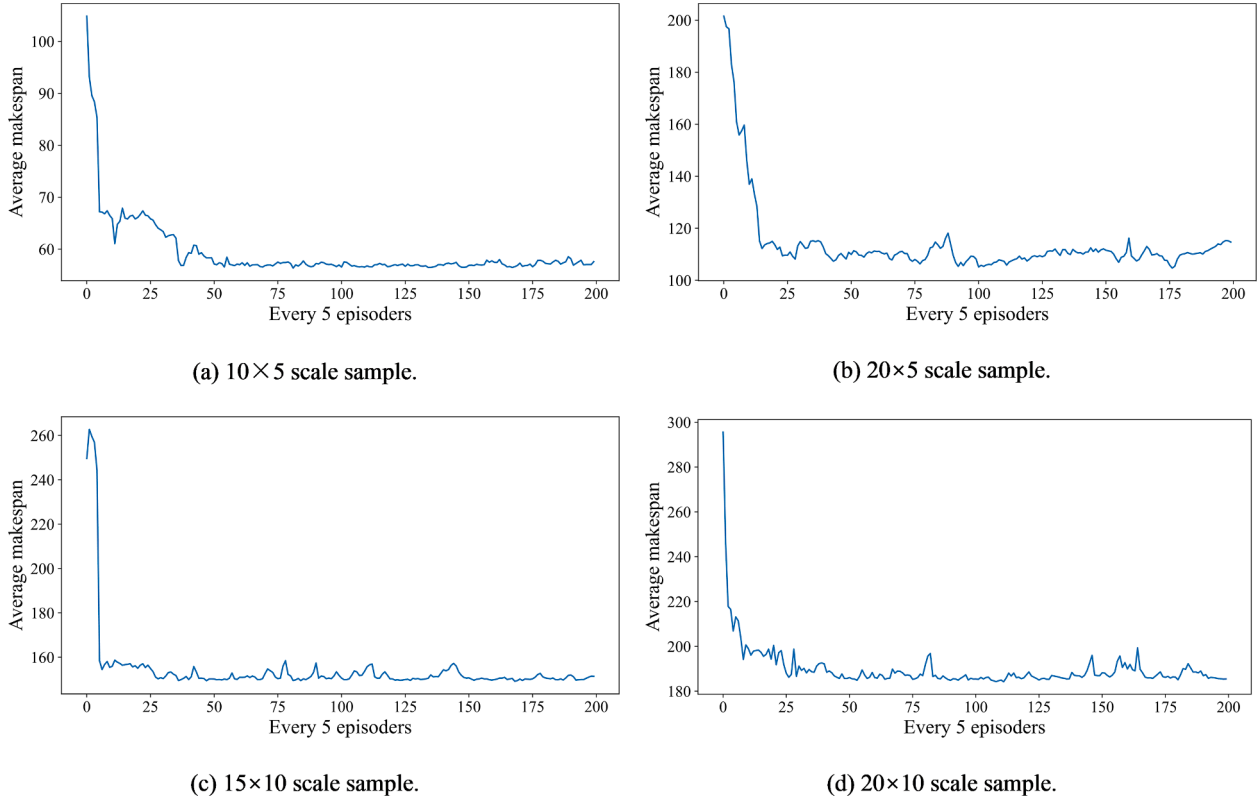


Fig. 9. Convergence curves of MAMHSAN under different training dataset scales.

Table 6

Average Gap and time of MAMHSAN on public datasets under different sizes for training datasets.

Size	Barnes		Brandimarte		Dauzere		Hurink(rdata)		Hurink(edata)		Hurink(vdata)	
	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)
10 × 5	10.73 %	2.12	4.47 %	1.53	5.12 %	3.18	7.63 %	1.75	9.24 %	1.84	1.63 %	1.84
20 × 5	22.98 %	2.08	12.99 %	1.52	10.28 %	3.23	15.6 %	1.77	21.02 %	1.78	12.63 %	1.78
15 × 10	17.57 %	2.08	12.56 %	1.41	8.66 %	3.18	15.6 %	1.83	16.78 %	1.71	8.29 %	1.72
20 × 10	31.93 %	2.11	13.34 %	1.48	15.03 %	3.36	22.32 %	1.84	28.36 %	1.84	8.8 %	1.81

dataset. In previous graph-based GNN and DLR methods, the computational cost increased with the size of the training set. From Table 6, it is apparent that although the sizes of various public datasets differ, MAMHSAN's solution time remains nearly the same regardless of the training set size used to solve them. This clearly shows that MAMHSAN can effectively handle FJSP of any size while achieving high-quality solutions with lower computational costs.

5.2.2. Performance on public benchmarks

To verify the effectiveness of the MAMHSAN, this subsection compared it with several state-of-the-art DRL-based methods and PDR methods using public datasets: Barnes, Brandimarte, Dauzere, and Hurink, respectively. The DRL-based methods proposed by Yuan et al. (2024), Song et al. (2023), and Lei et al. (2022). These PDR methods are described as FIFO, MWKR, and MOPNR, respectively. Moreover, we also included experimental results from effective metaheuristic methods for

Table 7

Comparison of average gap and average time on public datasets with various baselines.

Methods	Barnes		Brandimarte		Dauzere		Hurink(rdata)		Hurink(edata)		Hurink(vdata)	
	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)
IJA	2.40 %	21.03	8.50 %	15.43	6.10 %	180.8	3.90 %	19.72	4.60 %	15.24	2.70 %	17.85
RegGA	—	—	8.39 %	280.10	—	—	—	—	—	—	3.20 %	191.40
2SGA	—	—	3.17 %	57.60	—	—	—	—	—	—	0.39 %	51.43
SLGA	—	—	6.21 %	283.28	—	—	—	—	—	—	—	—
FIFO	27.91 %	0.019	28.98 %	0.017	15.00 %	0.036	17.38 %	0.018	19.89 %	0.016	7.14 %	0.019
MWKR	54.71 %	0.018	39.50 %	0.019	24.69 %	0.036	26.60 %	0.018	44.68 %	0.018	8.96 %	0.020
MOPNR	50.66 %	0.017	43.39 %	0.018	32.17 %	0.038	26.47 %	0.019	43.67 %	0.018	13.14 %	0.019
Song et al. (2023)	17.88 %	1.67	30.04 %	1.48	8.88 %	2.86	11.02 %	1.53	16.66 %	1.58	4.41 %	1.42
Kun Lei et al. (2022)	28.96 %	2.10	13.59 %	2.05	15.68 %	3.98	16.51 %	1.68	23.01 %	1.74	6.96 %	1.66
Yuan et al. (2024)	13.83 %	0.52	13.24 %	0.58	11.02 %	0.88	12.13 %	0.33	15.54 %	0.35	5.37 %	0.31
Ours	10.73 %	2.12	4.74 %	1.53	5.12 %	3.18	7.63 %	1.75	9.24 %	1.84	1.63 %	1.84

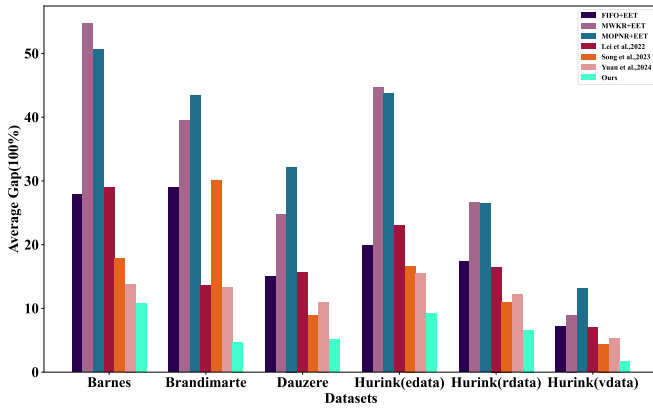


Fig. 10. Comparison chart with other DRL-based methods and PDRs.

comparisons, which are IJA, RegGA, 2SGA, and SLGA, respectively. It is worth mentioning that, based on the literature (Lei et al., 2022; Song et al., 2023; Yuan et al., 2024) and extensive experimental evidence of all DRL-based methods, using a 10×5 scale dataset as the training set can improve the performance of DRL-based methods on existing public datasets compared to using datasets of other scales. Therefore, we used a 10×5 dataset as the training set for all DRL-based methods. Meanwhile, to ensure fairness, all DRL-based methods adopt a random strategy during training and a greedy strategy during experimental testing on a public dataset, so that all DRL-based methods can obtain the optimal results of their respective algorithms.

In order to better demonstrate the performance of different algorithms in solving specific FJSP cases, we take the solution results of Google OR Tools as the optimal solution for each case, and then use Eq. (31) to calculate the gap between each method and the result, which we refer to as gap in the article. For Google OR Tools, we set the solution time for each FJSP instance to 30 min. In this study, we evaluated the average gap and average solution time of various methods across several public datasets, with the final results presented in Table 7. Specifically, for the Barnes dataset, the MAMHSAN achieved an average gap of 10.73 %, outperforming all other methods except for IJA, which recorded a gap of 2.40 %. The average gaps for the remaining methods were as follows: FIFO (27.91 %), MWKR (54.71 %), and MOPNR (50.66 %), Song et al. (2023) (17.88 %), Lei et al. (2022) (28.96 %), and Yuan et al. (2024) (13.83 %). These results indicate that the overall performance of these PDR methods is inferior to that of the DRL-based methods, with MAMHSAN and Yuan et al. (2024) from the DRL-based method leading the performance metrics. For the Brandimarte dataset, the MAMHSAN's average gap decreased to 4.74 %, again outperforming all algorithms except for 2SGA, which had an average gap of 3.17 %. The average gaps for the remaining methods were as follows: FIFO (28.98 %), MWKR (39.50 %), and MOPNR (43.39 %), Song et al. (2023) (30.40 %), Lei et al. (2022) (13.59 %), and Yuan et al. (2024) (13.24 %). Among these metaheuristic methods, IJA, RegGA, and SLGA had average gaps of 8.50 %, 8.39 %, and 6.21 %, respectively. It is evident that, among the DRL-based methods, all except for Song et al. (2023) significantly outperformed the PDR methods. Notably, the MAMHSAN average gap on this dataset even surpassed those of the IJA, RegGA, and SLGA, demonstrating its optimization capability. For the Dauzere dataset, the MAMHSAN recorded an average gap of 5.12 %, exceeding IJA's 6.10 % and leading among all algorithms. The average gaps for the remaining methods were as follows: FIFO (15.00 %), MWKR (24.69 %), and MOPNR (32.17 %), Song et al. (2023) (8.88 %), Lei et al. (2022) (15.68 %), and Yuan et al. (2024) (11.02 %), respectively. This analysis further confirms that the DRL-based methods continue to significantly outperform the PDR methods, with the MAMHSAN and Song et al. (2023) maintaining a leading position. For the Hurink (rdata) dataset, the MAMHSAN exhibits an average gap of 7.63 %, which is only surpassed

Table 8
Comparison of solution quality across all methods on Barnes.

Instances	Size	UB		FIFO		MWKR		MOPNR		Song et al. (2023)		Lei et al. (2022)		Yuan et al. (2024)		Ours	
		C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	
mt10x	10*11	918	—	1223	33.22 %	1253	36.49 %	1358	47.93 %	1096	19.39 %	1176	28.10 %	1084	18.08 %	1022	11.33 %
mt10xx	10*12	918	—	1256	36.82 %	1253	36.49 %	1374	49.67 %	1069	16.45 %	1176	28.10 %	1084	18.08 %	1022	11.33 %
mt10xxx	10*13	918	—	1250	36.17 %	1253	36.49 %	1416	54.25 %	1069	16.45 %	1176	28.10 %	1084	18.08 %	1022	11.33 %
mt10xy	10*12	906	—	1170	29.14 %	1252	38.19 %	1391	53.53 %	1090	20.31 %	1148	26.71 %	1055	16.45 %	985	8.72 %
mt10xyz	10*13	847	—	1058	24.91 %	1111	31.17 %	1292	52.54 %	1003	18.42 %	1180	39.32 %	1020	20.43 %	966	14.05 %
mt10c1	10*11	928	—	1207	30.06 %	1252	34.91 %	1353	45.80 %	1159	24.89 %	1240	33.62 %	1096	18.10 %	1004	8.19 %
mt10cc	10*12	910	—	1178	29.45 %	1121	23.19 %	1389	52.64 %	1100	20.88 %	1226	34.73 %	1068	17.36 %	980	7.69 %
setb4x	15*11	925	—	1140	23.24 %	1664	79.89 %	1322	42.92 %	1118	20.86 %	1210	30.81 %	1022	10.49 %	1035	11.89 %
setb4xx	15*12	925	—	1149	24.22 %	1691	82.81 %	1349	45.84 %	1119	20.97 %	1210	30.81 %	1015	9.73 %	1037	12.11 %
setb4xxx	15*13	925	—	1140	23.24 %	1664	79.89 %	1282	38.59 %	1119	20.97 %	1210	30.81 %	1040	12.43 %	1035	11.89 %
setb4xy	15*12	916	—	1125	22.82 %	1664	81.66 %	1372	49.78 %	1081	18.01 %	1111	21.29 %	1019	11.24 %	1021	11.46 %
setb4xyz	15*13	905	—	1082	19.56 %	1664	83.87 %	1355	49.72 %	1149	26.96 %	1095	20.99 %	1000	10.50 %	1000	10.50 %
setb4c9	15*11	919	—	1178	28.18 %	1589	72.91 %	1379	50.05 %	1073	16.76 %	1225	33.30 %	1037	12.84 %	1040	13.17 %
setb4cc	15*12	909	—	1234	35.75 %	1532	68.54 %	1363	49.94 %	1016	11.77 %	1180	29.81 %	993	9.24 %	1037	14.08 %
set5x	15*16	1201	—	1445	20.32 %	1756	46.21 %	1918	59.70 %	1377	14.65 %	1592	32.56 %	1363	13.49 %	1306	8.74 %
set5xxx	15*17	1199	—	1517	26.52 %	1731	44.37 %	1700	41.78 %	1377	14.85 %	1592	32.78 %	1344	12.09 %	1311	9.34 %
set5xxxx	15*18	1197	—	1590	32.83 %	1755	46.62 %	1852	54.72 %	1373	14.70 %	1592	33.00 %	1340	11.95 %	1311	9.52 %
set5xyx	15*17	1136	—	1404	23.59 %	1732	52.46 %	1728	52.11 %	1299	14.35 %	1438	26.58 %	1271	11.88 %	1230	8.27 %
set5xyz	15*18	1125	—	1464	30.13 %	1732	53.96 %	1592	41.51 %	1250	11.11 %	1298	15.38 %	1254	11.47 %	1252	11.29 %
set5c12	15*16	1174	—	1523	29.73 %	1953	66.35 %	1899	61.75 %	1390	18.40 %	1467	24.96 %	1346	14.65 %	1287	9.63 %
set5cc	15*17	1136	—	1436	26.41 %	1732	52.46 %	1921	69.10 %	1299	14.35 %	1438	26.58 %	1271	11.88 %	1259	10.83 %

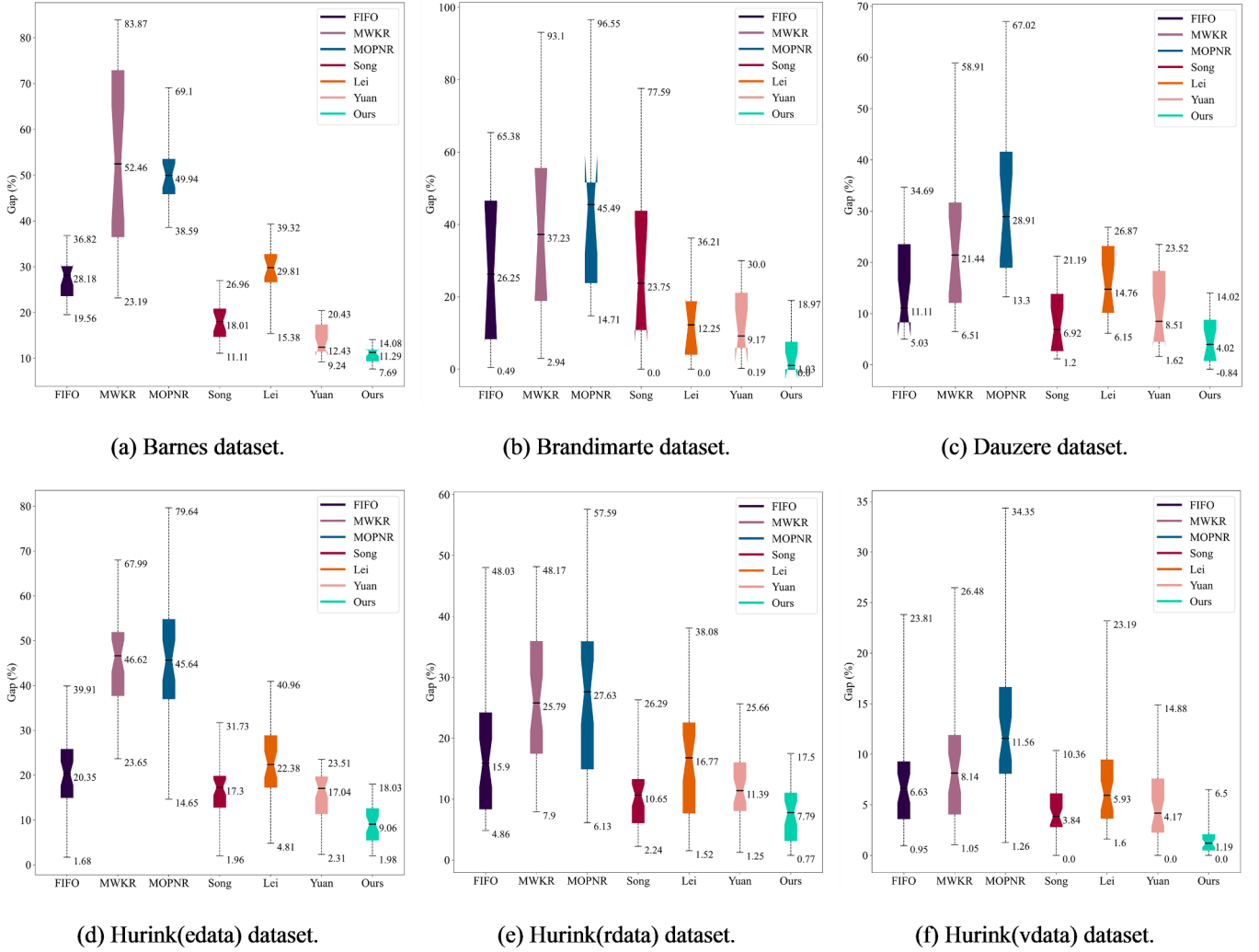


Fig. 11. Relative gaps of MAMHSAN against SOTA methods from baselines on public datasets.

by IJA's 3.90 %. In comparison, the average gaps for the remaining methods were as follows: FIFO (17.38 %), MWKR (26.60 %), and MOPNR (26.47 %), [Song et al. \(2023\)](#) (11.02 %), [Lei et al. \(2022\)](#) (16.51 %), and [Yuan et al. \(2024\)](#) (12.13 %). For the Hurink (edata) dataset, the MAMHSAN shows an average gap of 9.24 %, again only falling short of IJA's 4.60 %. The average gaps for the other methods were as follows: FIFO (19.89 %), MWKR (44.68 %), and MOPNR (43.67 %), [Song et al. \(2023\)](#) (16.66 %), [Lei et al. \(2022\)](#) (23.01 %), and [Yuan et al. \(2024\)](#) (15.54 %). In the case of the Hurink (vdata) dataset, the MAMHSAN records an average gap of 1.63 %, which is only exceeded by 2SGA's 0.39 %. The average gaps for the rest of the methods were as follows: FIFO (7.14 %), MWKR (8.96 %), and MOPNR (13.14 %), [Song et al. \(2023\)](#) (4.41 %), [Lei et al. \(2022\)](#) (6.96 %), and [Yuan et al. \(2024\)](#) (5.37 %). Among these metaheuristic methods, IJA and RegGA exhibit average gaps of 2.70 % and 3.20 %, respectively. Overall, DRL-based methods significantly outperform PDR methods in the Hurink (rdata) and Hurink (edata) datasets. The MAMHSAN ranks as the best among other DRL-based methods, followed by IJA and 2SGA. However, it is important to note that these methods typically require substantial computational time. In contrast, PDR methods require the least time and demonstrate good applicability. These DRL-based methods occupy the middle ground, often achieving high levels of performance within acceptable runtimes.

Among all DRL-based methods, [Yuan et al. \(2024\)](#) achieve the shortest processing time due to their use of a lightweight MLP to extract

the states from scheduling environments. In contrast, [Song et al. \(2023\)](#) employ the HGNN approach to extract features from the decision-making environment, while [Lei et al. \(2022\)](#) utilize multiple actions to make decisions for FJSP, resulting in relatively longer processing times for both. The MAMHSAN employs the MHSAN to extract features from the scheduling environment and incorporates multi-agents for joint decision-making. Consequently, the solution time of the MAMHSAN falls between those of [Song et al. \(2023\)](#) and [Lei et al. \(2022\)](#). However, objectively speaking, there is little overall difference in solution time, and high-quality solutions can be obtained within an acceptable time-frame. To gain a clearer understanding of the algorithm's performance across various datasets, we compare the FIFO, MWKR, and MOPNR methods of the PDRs with the methods proposed by ([Song et al., 2023](#)), ([Lei et al., 2022](#)), and ([Yuan et al., 2024](#)) within the DRL framework, as well as the average gap of these methods across different datasets. These comparisons are illustrated in [Fig. 10](#), which indicates that DRL-based methods demonstrate a stronger generalization ability than PDR-type methods across all datasets. Furthermore, among all the algorithms examined, the MAMHSAN exhibits the strongest generalization capability and the highest solution quality. Specifically, the performance of different algorithms in solving each instance for every dataset is analyzed in the following subsections.

5.2.3. Experimental results on the Barnes dataset

For the Barnes dataset, we selected all 21 instances as the verification

Table 9
Comparison of solution quality across all methods on Brandimarte.

Instances	Size	UB	FIFO		MWKR		MOPNR		Song et al. (2023)		Lei et al. (2022)		Yuan et al. (2024)		Ours	
			C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap
mk01	10*6	40	49	22.50 %	51	27.50 %	57	42.50 %	49	22.50 %	47	17.50 %	44	10.00 %	40	0.00 %
mk02	10*6	26	43	65.38 %	41	57.69 %	41	57.69 %	43	65.38 %	30	15.38 %	31	19.23 %	28	7.69 %
mk03	15*8	204	205	0.49 %	210	2.94 %	234	14.71 %	216	5.88 %	204	0.00 %	211	3.43 %	204	0.00 %
mk04	15*8	60	78	30.00 %	99	65.00 %	90	50.00 %	75	25.00 %	76	26.67 %	78	30.00 %	67	11.67 %
mk05	15*4	173	184	6.36 %	202	16.76 %	211	21.97 %	190	9.83 %	178	2.89 %	183	5.78 %	176	1.73 %
mk06	10*10	58	92	58.62 %	112	93.10 %	114	96.55 %	103	77.59 %	79	36.21 %	74	27.59 %	69	18.97 %
mk07	20*5	144	214	48.61 %	215	49.31 %	219	52.08 %	212	47.22 %	152	5.56 %	156	8.33 %	144	0.00 %
mk08	20*10	523	541	3.44 %	579	10.71 %	631	20.65 %	523	0.00 %	541	3.44 %	524	0.19 %	523	0.00 %
mk09	20*10	307	350	14.01 %	384	25.08 %	397	29.32 %	349	13.68 %	335	9.12 %	326	6.19 %	308	0.33 %
mk10	20*15	198	278	40.40 %	291	46.97 %	294	48.48 %	264	33.33 %	236	19.19 %	241	21.72 %	212	7.07 %

set. The scales of these instances include 10×11 , 10×12 , 10×13 , 10×14 , 15×12 , 15×13 , 15×16 , 15×17 , and 15×18 , respectively. In the experiment, we employed a policy network trained on random instances of a 10×5 scale to address these FJSP instances. To ensure fairness across all DRL-based methods, we utilized a greedy strategy to obtain the final results. Additionally, we applied three PDR methods—FIFO, MWKR, and MOPNR—to solve all FJSP instances. Table 8 presents the exact results of all PDR methods and DRL-based methods for each calculation example in the Barnes dataset. Furthermore, to provide a more intuitive comparison of solution quality across different algorithms, we used the results from the exact method (*i.e.*, Google OR-Tools) as optimal results for specific instances. This was combined with the calculation method of RPD as outlined in Eq. (31) to compare the performance differences of each algorithm on each instance. In the following tables, we denote these values as 'gap'.

As shown in Table 8, the MAMHSAN outperforms other comparative algorithms across almost all instances within the Barnes dataset. Specifically, for the PDR methods, the gap distribution range for the FIFO method across all instances is between 19.56 % and 36.82 %, while the MWKR method exhibits a gap distribution range of 23.19 % to 83.87 %, and the MOPNR method ranges from 38.59 % to 69.1 %. Among these PDR methods, FIFO demonstrates superior generalization within the Barnes dataset. For the DRL-based methods, the gap distribution range for the Song et al. (2023) across all instances is 11.11 % to 29.96 %, the range for Lei et al. (2022) is 15.38 % to 39.32 %, and the range for Yuan et al. (2024) is 9.24 % to 20.43 %. In contrast, the gap distribution range for the MAMHSAN across all instances is 7.69 % to 14.08 %. To enhance the clarity and intuitiveness regarding the differences among different algorithms, we display the distribution of gaps for each algorithm, as shown in Fig. 11 (a). The median solution quality for these methods: FIFO (28.18 %), MWKR (52.64 %), and MOPNR (49.94 %), Song et al. (2023) (18.01 %), Lei et al. (2022) (29.81 %), and Yuan et al. (2024) (12.43 %), MAMHSAN (11.29 %). By integrating the findings presented in Table 7, which compares the average gap and average time across different public datasets and various baseline approaches. From Fig. 11 (a), it is evident that the method proposed by Yuan et al. (2024) and this study exhibit a stronger generalization ability on the Barnes dataset. In terms of solution quality, our proposed MAMHSAN significantly outperforms all comparative algorithms.

5.2.4. Experimental results on the Brandimarte dataset

In this subsection, we utilize the typical FJSP dataset from Brandimarte, specifically selecting all instances (*i.e.*, MK01-MK10) as a verification set. The scales of these public instances include 10×6 , 10×10 , 15×4 , 15×8 , 20×5 , 20×10 , and 20×15 , respectively. For the following experiments, we employed a policy network trained on random instances of the 20×10 scale to address these FJSP instances. To ensure fairness, we adopted a greedy strategy for obtaining the final results across all DRL-based methods. Additionally, we implemented three PDRs, *i.e.*, FIFO, MWKR, and MOPNR, to solve all FJSP instances. Table 9 presents the exact results of all PDRs and DRL-based methods for each instance in the Barnes dataset. Furthermore, to provide a clearer comparison of the solution quality derived from different algorithms, we used the results obtained from Google OR-Tools as the optimal results for instances of different sizes. All comparison analyses are the same as in subsection 5.2.3. As shown in Table 9, the MAMHSAN outperforms other algorithms across all instances of the Brandimarte dataset. Specifically, for PDR methods, the gap distribution range for the FIFO method in all instances is between 0.49 % and 65.38 %, while the MWKR method exhibits a gap distribution range of 2.94 % to 93.1 %, and the MOPNR method ranges from 14.71 % to 96.55 %. Among these PDR methods, FIFO demonstrates superior generalization in the Brandimarte dataset. For the DRL-based method, the gap distribution range across all instances for the Song et al. (2023) dataset is 0.0 % to 77.59 %, for Lei et al. (2022) it is 0.0 % to 36.21 %, and for Yuan et al. (2024) it is 0.19 % to 30.0 %. In contrast, the gap distribution range of the MAMHSAN

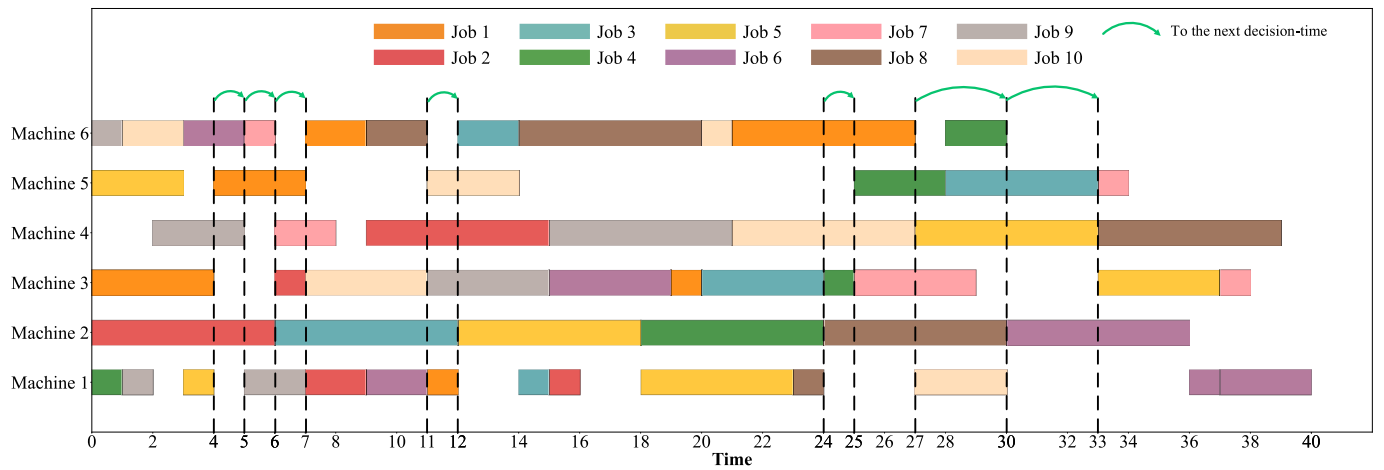


Fig. 12. Gantt chart of the results of the MAMHSAN on Brandimarte's MK01.

across all instances is 0.0 % to 18.97 %. Notably, for the instances MK01, MK03, MK07, and MK08, we obtained results identical to those produced by the exact method, i.e., Google OR-Tools. The Gantt charts for MK01, MK03, and MK08 are displayed in Figs. 13, 14, and 15, respectively. In Fig. 12 and Fig. 14, the dotted lines and green arrows indicate

that the DDT agent transfers $T(t)$ to the $T'(t)$ during the decision-making process. The results presented in the Gantt charts demonstrate that, through the effective collaboration of both DDT agent and SSA agent, the MAMHSAN achieved the best solutions for MK01 and MK03. In contrast, the final results for MK08, illustrated in Fig. 14, reveal that the

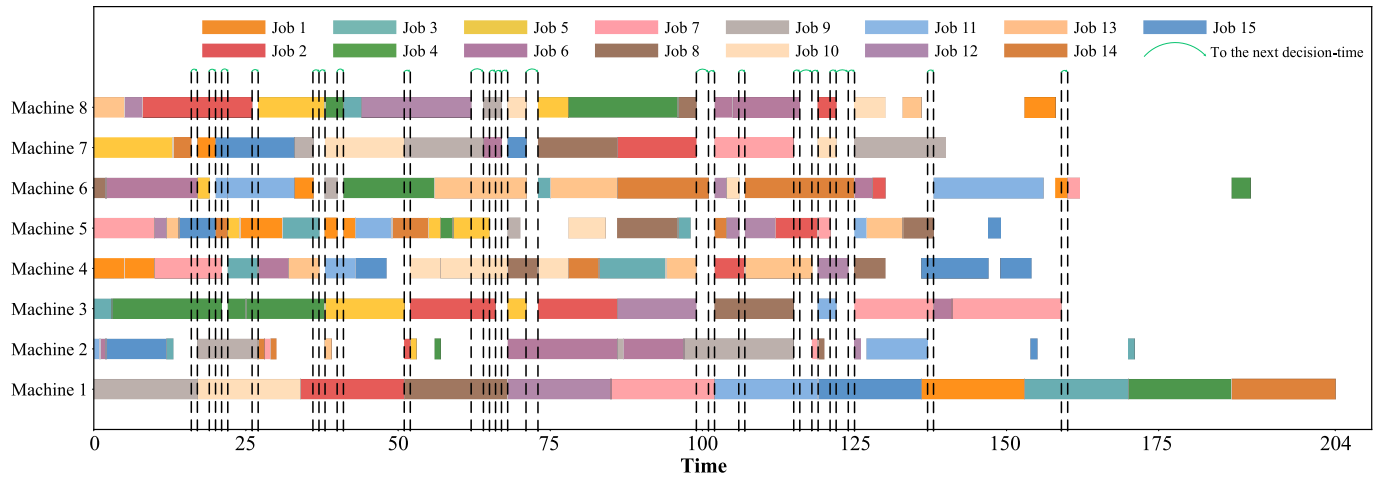


Fig. 13. Gantt chart of the results of the MAMHSAN on Brandimarte's MK03.

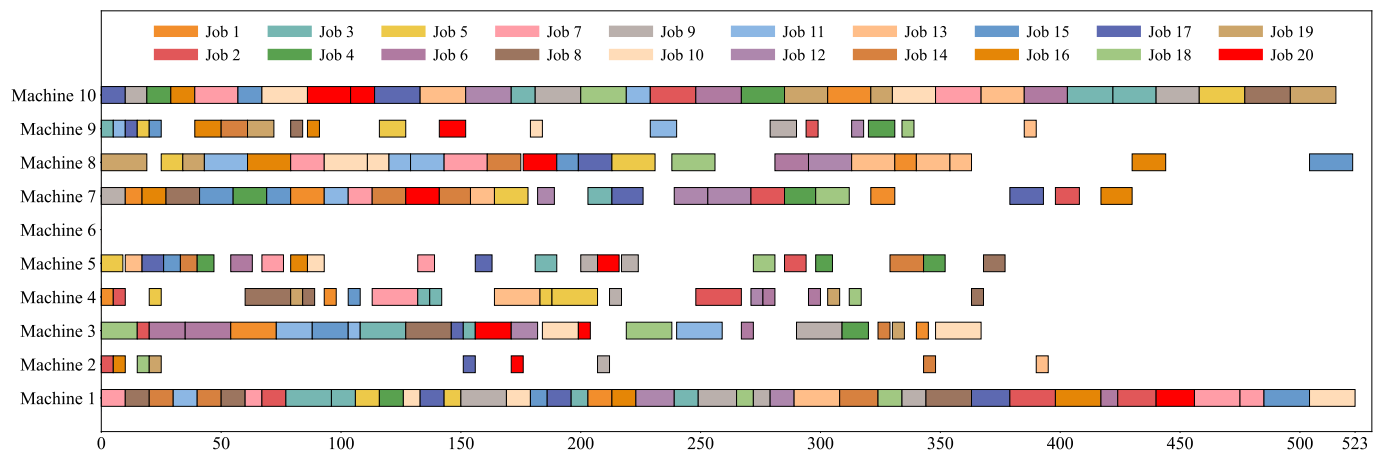


Fig. 14. Gantt chart of the results of the MAMHSAN on Brandimarte's MK08.

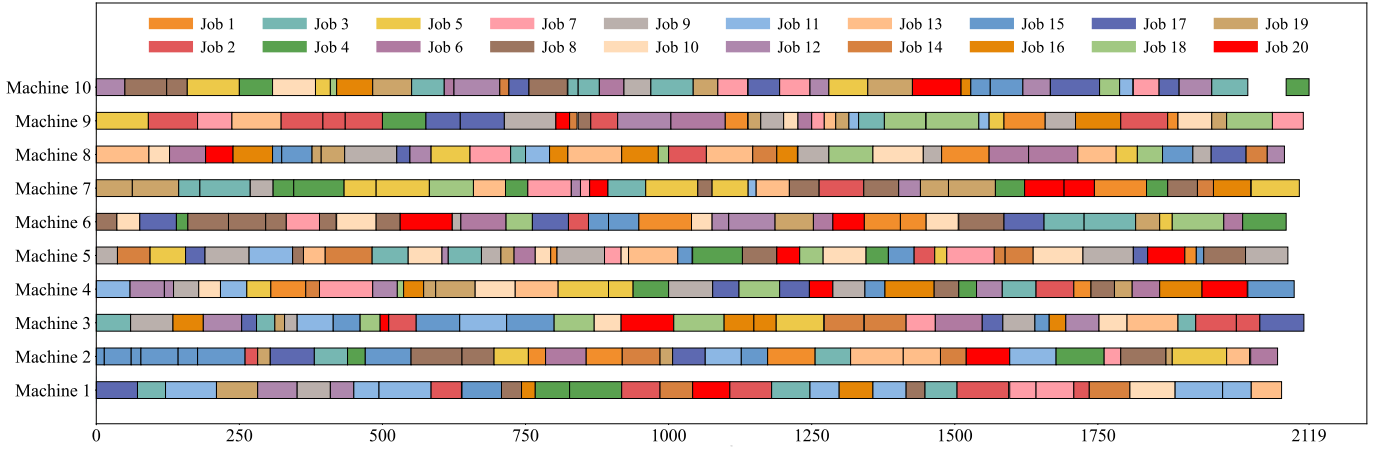


Fig. 15. Gantt chart of the results of the MAMHSAN on Dauzere's 18a.

lengthy overall processing time made it impractical to display the decision-making process of the DDT agent in the figure. Interested readers can access the specific scheduling results via the source code provided. It is noteworthy that [Lei et al. \(2022\)](#) and [Song et al. \(2023\)](#) also obtained solutions of comparable quality for MK03 and MK08; however, they were unable to achieve the same results as our study for MK01, MK03, MK07, and MK08 simultaneously. This discrepancy underscores the effectiveness of the MAMHSAN and its strong optimization capabilities, indicating that the MD-MDP significantly enhances solution quality.

To enhance the clarity and intuitiveness regarding the differences among different algorithms, we display the distribution of gaps for each algorithm in [Fig. 11 \(b\)](#). The median solution quality for the methods: FIFO (26.25 %), MWKR (37.23 %), and MOPNR (45.49 %), [Song et al. \(2023\)](#) (23.26 %), [Lei et al. \(2022\)](#) (12.25 %), and [Yuan et al. \(2024\)](#) (9.17 %), MAMHSAN (1.03 %), respectively. Notably, on the Brandimarte dataset, the DRL-based methods demonstrate superior performance compared to the PDR methods, exhibiting enhanced overall generalization ability as well. Among all DRL-based methods analyzed, the MAMHSAN leads in both generalization ability and solution quality.

5.2.5. Experimental results on the Dauzere dataset

For the Dauzere dataset, we selected all 18 calculation instances (01a-18a) as the verification set. The scales of these FJSP instances include 10×5 , 10×8 , and 20×10 , respectively. In the experiment, we employed a policy network trained on random instances of the 15×10 scale to address these FJSP instances. To ensure fairness across all DRL-based methods, we utilized the greedy strategy to obtain the final results. Additionally, we applied three PDR methods (FIFO, MWKR, and MOPNR) to solve all instances for FJSP. [Table 10](#) presents the results of these PDRs and DRL-based methods for each instance in the Barnes dataset. To provide a clearer view of the differences in solution quality among various algorithms, we employed the results from Google OR-Tools as the optimal results for specific instances. The comparisons for each algorithm across the instances are the same as in subsection 5.2.3. As illustrated in [Table 10](#), the MAMHSAN outperforms other algorithms across all instances of the Dauzere dataset. Specifically, for the PDR methods, the gap distribution range for the FIFO method spans from 5.03 % to 34.69 %, while the MWKR method ranges from 6.51 % to 58.91 %, and the MOPNR method ranges from 13.3 % to 67.02 %. Among these PDR methods, the FIFO demonstrates superior generalization on the Dauzere dataset. For DRL-based methods, the gap distribution range for [Song et al. \(2023\)](#) is from 1.2 % to 1.19 %, for [Lei et al. \(2022\)](#) is from 6.15 % to 26.87 %, and for [Yuan et al. \(2024\)](#) it ranges

from 1.62 % to 23.52 %. In contrast, the gap distribution range for the MAMHSAN is from -0.84 % to 14.02 %. Notably, the MAMHSAN achieved results that surpassed the Google OR-Tools in instances 12a and 18a, with gaps of -0.1 % and -0.84 %, respectively, thereby significantly demonstrating the effectiveness and optimization capabilities of the MAMHSAN. [Fig. 15](#) presents the Gantt chart results for instance 18a. Due to its lengthy processing time, the decision-making of the DDT agent depicted in the figure is challenging to display. Interested readers can access detailed scheduling results through the provided source code. To enhance the clarity and intuitiveness regarding the differences among different algorithms, we display the distribution of gaps for each algorithm in [Fig. 11 \(c\)](#). The median solution quality for methods: FIFO (11.11 %), MWKR (21.44 %), and MOPNR (28.91 %), [Song et al. \(2023\)](#) (6.92 %), [Lei et al. \(2022\)](#) (14.76 %), and [Yuan et al. \(2024\)](#) (8.51 %), MAMHSAN (4.02 %). By considering the results from [Table 10](#) and [Fig. 11 \(c\)](#), it is evident that the method proposed by [Yuan et al. \(2024\)](#), along with the MAMHSAN presented in this study, demonstrates superior generalization ability on the Dauzere dataset. Furthermore, in terms of solution quality, the algorithms introduced in this study significantly outperform all comparative algorithms.

5.2.6. Experimental results on the Hurink dataset

In this subsection, three sub-datasets were selected from the Hurink dataset, each exhibiting unique characteristics, namely edata, rdata, and vdata. In edata, the characteristics include a limited number of operations that can be assigned to various machines, whereas rdata allows most operations to be assigned to a small number of different machines. In contrast, the vdata enables each operation to be assigned to multiple machines. For these three sub-datasets, we selected the calculation instances la1-la40 contained within them as the verification set. The scales of the instances include 10×5 , 10×10 , 15×5 , 15×10 , 15×15 , 20×5 , 20×10 and 30×10 . In the following experiment, we utilized a policy network trained on random instances of sizes 10×5 and 15×10 to address these FJSP instances. To ensure fairness, we employed a greedy strategy to obtain the final results for all DRL-based methods. Additionally, we applied the same FIFO, MWKR, and MOPNR to solve all FJSP instances. [Tables 11, 12, and 13](#) present the results of both PDR methods and DRL-based methods for each calculation instance across this sub-dataset. Furthermore, we utilized the results from Google OR-Tools as the baselines for each specific instance. The comparisons for algorithms across instances are consistent with those in subsection 5.2.3. As shown in [Table 11](#), the MAMHSAN significantly outperforms all PDRs and DRL-based methods on the edata sub-dataset from the Hurink dataset. For the PDR methods, the gap distribution range of the FIFO

Table 10
Comparison of solution quality across all methods on Dauzere.

Instances	Size	UB		FIFO		MWKR		MOPNR		Song et al. (2023)		Lei et al. (2022)		Yuan et al. (2024)		Ours	
		C_{max}	Gap	C_{max}	Gap	C_{max}	Gap	C_{max}	Gap	C_{max}	Gap	C_{max}	Gap	C_{max}	Gap	C_{max}	Gap
01a	10*5	2518	—	3138	24.62 %	3352	33.12 %	3545	40.79 %	2847	13.07 %	3109	23.47 %	2947	17.04 %	2754	9.37 %
02a	10*5	2231	—	2416	8.29 %	2753	23.40 %	3084	38.23 %	2478	11.07 %	2603	16.67 %	2495	11.83 %	2390	7.13 %
03a	10*5	2229	—	2359	5.83 %	2456	10.18 %	2650	18.89 %	2294	2.92 %	2366	6.15 %	2320	4.08 %	2258	1.30 %
04a	10*5	2503	—	3109	24.21 %	3188	27.37 %	3511	41.87 %	2856	14.10 %	3093	23.57 %	3037	21.33 %	2681	7.11 %
05a	10*5	2216	—	2525	13.94 %	2684	21.12 %	2887	30.28 %	2473	11.60 %	2573	16.11 %	2496	12.64 %	2348	5.96 %
06a	10*5	2203	—	2410	9.40 %	2478	12.48 %	2496	13.30 %	2262	2.68 %	2508	13.84 %	2286	3.77 %	2211	0.36 %
07a	15*8	2283	—	2897	26.89 %	3628	58.91 %	3813	67.02 %	2736	19.84 %	2857	25.14 %	2820	23.52 %	2603	14.02 %
08a	15*8	2069	—	2321	12.18 %	2622	26.73 %	2639	27.55 %	2191	5.90 %	2386	15.32 %	2259	9.18 %	2163	4.54 %
09a	15*8	2066	—	2170	5.03 %	2313	11.96 %	2391	15.73 %	2097	1.50 %	2209	6.92 %	2155	4.31 %	2078	0.58 %
10a	15*8	2291	—	2786	21.61 %	3386	47.80 %	3501	52.82 %	2635	15.02 %	2805	22.44 %	2721	18.77 %	2593	13.18 %
11a	15*8	2063	—	2401	16.38 %	2512	21.76 %	2804	35.92 %	2227	7.95 %	2356	14.20 %	2206	6.93 %	2135	3.49 %
12a	15*8	2034	—	2203	8.31 %	2228	9.54 %	2424	19.17 %	2090	2.75 %	2186	7.47 %	2157	6.05 %	2032	-0.10 %
13a	20*10	2260	—	3044	34.69 %	3320	46.90 %	3500	54.87 %	2739	21.19 %	2811	24.38 %	2755	21.90 %	2505	10.84 %
14a	20*10	2167	—	2347	8.31 %	2510	15.83 %	2660	22.75 %	2218	2.35 %	2353	8.58 %	2277	5.08 %	2214	2.17 %
15a	20*10	2167	—	2325	7.29 %	2308	6.51 %	2471	14.03 %	2193	1.20 %	2385	10.06 %	2202	1.62 %	2170	0.14 %
16a	20*10	2255	—	2821	25.10 %	3336	47.94 %	3313	46.92 %	2723	20.75 %	2861	26.87 %	2680	18.85 %	2522	11.84 %
17a	20*10	2141	—	2356	10.04 %	2481	15.88 %	2653	23.91 %	2224	3.88 %	2365	10.46 %	2309	7.85 %	2169	1.31 %
18a	20*10	2137	—	2308	8.00 %	2289	7.11 %	2458	15.02 %	2185	2.25 %	2363	10.58 %	2216	3.70 %	2119	-0.84 %

across all instances is between 1.68 % and 39.91 %, and MWKR exhibits a gap distribution range of 23.65 % to 67.99 %, and MOPNR ranges from 14.65 % to 79.64 %. Among these PDRs, FIFO demonstrates superior generalization for the edata sub-dataset. Notably, the results of the FIFO on the instance la06 within the edata sub-dataset surpass those of all other methods. For the DRL-based methods, the gap distribution range across all instances for the Song et al. (2023) method is 1.96 % to 37.73 %, for the Lei et al. (2022) method it is 4.81 % to 40.96 %, and for the Yuan et al. (2024) method it is 2.31 % to 23.51 %. In contrast, the gap distribution range for the MAMHSAN across all instances is 1.98 % to 18.03 %. To enhance the clarity regarding the differences among different algorithms, we display the distribution of gaps for each algorithm in Fig. 11 (d). The median solution quality for the methods is as follows: FIFO (20.35 %), MWKR (46.62 %), and MOPNR (45.64 %), Song et al. (2023) (17.3 %), Lei et al. (2022) (23.38 %), and Yuan et al. (2024) (17.04 %), MAMHSAN (9.06 %). It is evident that the MAMHSAN exhibits significantly better generalization ability compared to the other algorithms, and in terms of solution quality, it consistently outperforms almost all instances.

As depicted in Table 12, these results on the sub-dataset rdata from the Hurink dataset demonstrate that the MAMHSAN outperforms both the PDR and DRL-based methods in most instances. For the PDR methods, the gap distribution range for the FIFO method across all instances spans from 4.86 % to 48.03 %, while the MWKR method ranges from 7.9 % to 48.17 %, and the MOPNR method extends from 6.13 % to 57.79 %. Notably, among these PDR methods, FIFO exhibits superior generalization in solving the sub-dataset rdata. In contrast, for the DRL-based methods, the gap distribution ranges across all instances are as follows: for Song et al. (2023), 2.24 % to 26.29 %; for Lei et al. (2022), 1.52 % to 38.08 %; and for Yuan et al. (2024), 1.25 % to 25.66 %. The gap distribution range for the MAMHSAN across all instances is notably narrower, at 0.77 % to 17.5 %. To enhance the clarity and intuitiveness regarding the differences among different algorithms, we display the distribution of gaps for each algorithm in Fig. 11 (e). The median solution quality for the methods: FIFO (15.9 %), MWKR (25.79 %), and MOPNR (27.63 %), Song et al. (2023) (10.65 %), Lei et al. (2022) (16.77 %), and Yuan et al. (2024) (11.39 %), MAMHSAN (7.79 %). It is observed that MAMHSAN exhibits superiority with a more concentrated overall solution quality compared to all other algorithms.

As revealed in Table 13, for the sub-dataset vdata of the Hurink dataset, MAMHSAN significantly outperforms all PDR and DRL-based methods in almost all results, with only a few exceptions. For the PDR methods, the gap distribution ranges are as follows: the FIFO method exhibits a range of 0.95 % to 23.81 %, MWKR ranges from 1.05 % to 26.48 %, and MOPNR displays a range of 1.26 % to 34.35 %. Among these PDRs, FIFO demonstrates superior generalization within the sub-dataset vdata. In terms of the DRL-based methods, the gap distribution ranges are: Song et al. (2023) ranges from 0.0 % to 10.36 %, Lei et al. (2022) from 1.6 % to 23.19 %, and Yuan et al. (2024) from 0 % to 14.88 %. In contrast, MAMHSAN achieves a distribution range of 0.0 % to 6.5 % across all instances. Notably, the MAMHSAN produced superior results on la16, la17, la18, la20, la38, and la40, similar to those obtained by using Google OR-Tools. To enhance the clarity and intuitiveness regarding the differences among different algorithms, we display the distribution of gaps for each algorithm in Fig. 11 (f). The median solution quality for these methods is as follows: FIFO (6.63 %), MWKR (8.14 %), and MOPNR (11.56 %), Song et al. (2023) (3.84 %), Lei et al. (2022) (5.93 %), and Yuan et al. (2024) (4.17 %), MAMHSAN (1.19 %). It is concluded that the MAMHSAN exhibits better generalization ability and solution stability than all comparative algorithms and provides a more concentrated quality distribution of solutions. As indicated in Table 13, it has consistently achieved the best results across all instances with different scales, corroborated by the same results obtained from Google OR-Tools, which thoroughly demonstrates the optimization capability of the MAMHSAN.

Table 11

Comparison of solution quality accross all methods on Hurink (edata).

Instances	Size	UB		FIFO		MWKR		MOPNR		Song et al. (2023)		Lei et al. (2022)		Yuan et al. (2024)		Ours	
		$C_{\max}$	Gap	$C_{\max}$	Gap	$C_{\max}$	Gap	$C_{\max}$	Gap	$C_{\max}$	Gap	$C_{\max}$	Gap	$C_{\max}$	Gap	$C_{\max}$	Gap
la01	10*5	609	—	722	18.56 %	866	42.20 %	882	44.83 %	688	12.97 %	712	16.91 %	658	8.05 %	639	4.93 %
la02	10*5	655	—	797	21.68 %	982	49.92 %	992	51.45 %	851	29.92 %	875	33.59 %	809	23.51 %	692	5.65 %
la03	10*5	550	—	691	25.64 %	692	25.82 %	988	79.64 %	657	19.45 %	714	29.82 %	660	20.00 %	600	9.09 %
la04	10*5	568	—	664	16.90 %	872	53.52 %	836	47.18 %	660	16.20 %	711	25.18 %	669	17.78 %	641	12.85 %
la05	10*5	503	—	593	17.89 %	737	46.52 %	599	19.09 %	530	5.37 %	621	23.46 %	593	17.89 %	520	3.38 %
la06	15*5	833	—	847	1.68 %	1030	23.65 %	955	14.65 %	926	11.16 %	946	13.57 %	951	14.17 %	870	4.44 %
la07	15*5	762	—	969	27.17 %	1065	39.76 %	1063	39.50 %	910	19.42 %	1010	32.55 %	896	17.59 %	844	10.76 %
la08	15*5	845	—	949	12.31 %	1267	49.94 %	1162	37.51 %	969	14.67 %	932	10.30 %	947	12.07 %	867	2.60 %
la09	15*5	878	—	934	6.38 %	1229	39.98 %	1085	23.58 %	1046	19.13 %	964	9.79 %	946	7.74 %	913	3.99 %
la10	15*5	866	—	934	7.85 %	1167	34.76 %	1032	19.17 %	883	1.96 %	1016	17.32 %	886	2.31 %	885	2.19 %
la11	20*5	1103	—	1190	7.89 %	1451	31.55 %	1274	15.50 %	1172	6.26 %	1285	16.50 %	1213	9.97 %	1144	3.72 %
la12	20*5	960	—	1032	7.50 %	1282	33.54 %	1154	20.21 %	1001	4.27 %	1143	19.06 %	1065	10.94 %	979	1.98 %
la13	20*5	1053	—	1150	9.21 %	1392	32.19 %	1270	20.61 %	1215	15.38 %	1181	12.16 %	1164	10.54 %	1103	4.75 %
la14	20*5	1123	—	1306	16.30 %	1447	28.85 %	1487	32.41 %	1191	6.06 %	1177	4.81 %	1234	9.88 %	1160	3.29 %
la15	20*5	1111	—	1308	17.73 %	1541	38.70 %	1389	25.02 %	1301	17.10 %	1362	22.59 %	1297	16.74 %	1204	8.37 %
la16	10*10	892	—	1087	21.86 %	1351	51.46 %	1414	58.52 %	1059	18.72 %	1046	17.26 %	1051	17.83 %	973	9.08 %
la17	10*10	707	—	813	14.99 %	1039	46.96 %	1016	43.71 %	781	10.47 %	949	34.23 %	773	9.34 %	786	11.17 %
la18	10*10	842	—	952	13.06 %	1089	29.33 %	1153	36.94 %	944	12.11 %	970	15.20 %	934	10.93 %	911	8.19 %
la19	10*10	796	—	981	23.24 %	1100	38.19 %	1234	55.03 %	950	19.35 %	1001	25.75 %	972	22.11 %	868	9.05 %
la20	10*10	857	—	1102	28.59 %	1317	53.68 %	1199	39.91 %	1007	17.50 %	1102	28.59 %	1004	17.15 %	942	9.92 %
la21	15*10	1017	—	1282	26.06 %	1363	34.02 %	1594	56.74 %	1243	22.22 %	1262	24.09 %	1256	23.50 %	1172	15.24 %
la22	15*10	882	—	1234	39.91 %	1310	48.53 %	1208	36.96 %	977	10.77 %	1242	40.82 %	1070	21.32 %	1041	18.03 %
la23	15*10	950	—	1156	21.68 %	1396	46.95 %	1394	46.74 %	1131	19.05 %	1184	24.63 %	1137	19.68 %	1026	8.00 %
la24	15*10	909	—	1186	30.47 %	1527	67.99 %	1360	49.61 %	1112	22.33 %	1204	32.45 %	1086	19.47 %	1040	14.41 %
la25	15*10	941	—	1224	30.07 %	1401	48.88 %	1629	73.11 %	1139	21.04 %	1224	30.07 %	1102	17.11 %	1063	12.96 %
la26	20*10	1125	—	1372	21.96 %	1647	46.40 %	1674	48.80 %	1338	18.93 %	1478	31.38 %	1348	19.82 %	1266	12.53 %
la27	20*10	1186	—	1488	25.46 %	1863	57.08 %	1916	61.55 %	1485	25.21 %	1514	27.66 %	1423	19.98 %	1360	14.67 %
la28	20*10	1149	—	1464	27.42 %	1769	53.96 %	1609	40.03 %	1441	25.41 %	1375	19.67 %	1344	16.97 %	1321	14.97 %
la29	20*10	1118	—	1394	24.69 %	1741	55.72 %	1807	61.63 %	1353	21.02 %	1345	20.30 %	1359	21.56 %	1303	16.55 %
la30	20*10	1204	—	1550	28.74 %	1846	53.32 %	1863	54.73 %	1459	21.18 %	1471	22.18 %	1410	17.11 %	1398	16.11 %
la31	30*10	1539	—	1836	19.30 %	2170	41.00 %	2153	39.90 %	1720	11.76 %	1936	25.80 %	1755	14.04 %	1695	10.14 %
la32	30*10	1698	—	1967	15.84 %	2530	49.00 %	2490	46.64 %	1962	15.55 %	2043	20.32 %	1904	12.13 %	1841	8.42 %
la33	30*10	1547	—	1863	20.43 %	2396	54.88 %	2228	44.02 %	1776	14.80 %	1811	17.07 %	1785	15.38 %	1640	6.01 %
la34	30*10	1604	—	1872	16.71 %	2183	36.10 %	2349	46.45 %	1821	13.53 %	1917	19.51 %	1817	13.28 %	1708	6.48 %
la35	30*10	1736	—	1970	13.48 %	2547	46.72 %	2498	43.89 %	1967	13.31 %	1996	14.98 %	2080	19.82 %	1864	7.37 %
la36	15*15	1162	—	1518	30.64 %	1675	44.15 %	1775	52.75 %	1334	14.80 %	1638	40.96 %	1295	11.45 %	1286	10.67 %
la37	15*15	1397	—	1602	14.67 %	1987	42.23 %	2177	55.83 %	1650	18.11 %	1645	17.75 %	1531	9.59 %	1560	11.67 %
la38	15*15	1144	—	1376	20.28 %	1810	58.22 %	1690	47.73 %	1507	31.73 %	1355	18.44 %	1370	19.76 %	1327	16.00 %
la39	15*15	1184	—	1490	25.84 %	1757	48.40 %	1884	59.12 %	1547	30.66 %	1659	40.12 %	1412	19.26 %	1317	11.23 %
la40	15*15	1150	—	1447	25.83 %	1877	63.22 %	1798	56.35 %	1353	17.65 %	1422	23.65 %	1311	14.00 %	1251	8.78 %

Table 12
Comparison of solution quality accross all methods on Hurink (rdata).

Instances	Size	UB		FIFO		MWKR		MOPNR		Song et al. (2023)		Lei et al. (2022)		Yuan et al. (2024)		Ours	
		C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap
la01	10*5	571	—	689	20.67 %	709	24.17 %	687	20.32 %	616	7.88 %	690	20.84 %	663	16.11 %	595	4.20 %
la02	10*5	530	—	668	26.04 %	694	30.94 %	695	31.13 %	598	12.83 %	648	22.26 %	666	25.66 %	583	10.00 %
la03	10*5	478	—	538	12.55 %	623	30.33 %	565	18.20 %	507	6.07 %	528	10.46 %	525	9.83 %	484	1.26 %
la04	10*5	502	—	601	19.72 %	633	26.10 %	624	24.30 %	533	6.18 %	531	5.78 %	573	14.14 %	531	5.78 %
la05	10*5	457	—	502	9.85 %	540	18.16 %	507	10.94 %	504	10.28 %	516	12.91 %	500	9.41 %	469	2.63 %
la06	15*5	799	—	857	7.26 %	912	14.14 %	848	6.13 %	850	6.38 %	850	6.38 %	856	7.13 %	814	1.88 %
la07	15*5	750	—	798	6.40 %	846	12.80 %	908	21.07 %	787	4.93 %	786	4.80 %	825	10.00 %	772	2.93 %
la08	15*5	765	—	821	7.32 %	960	25.49 %	829	8.37 %	804	5.10 %	803	4.97 %	820	7.19 %	794	3.79 %
la09	15*5	853	—	924	8.32 %	1068	25.21 %	940	10.20 %	875	2.58 %	866	1.52 %	885	3.75 %	873	2.34 %
la10	15*5	804	—	846	5.22 %	973	21.02 %	920	14.43 %	832	3.48 %	885	10.07 %	870	8.21 %	835	3.86 %
la11	20*5	1071	—	1123	4.86 %	1157	8.03 %	1156	7.94 %	1135	5.98 %	1105	3.17 %	1131	5.60 %	1091	1.87 %
la12	20*5	936	—	1004	7.26 %	1050	12.18 %	1086	16.03 %	957	2.24 %	988	5.56 %	977	4.38 %	953	1.82 %
la13	20*5	1038	—	1089	4.91 %	1120	7.90 %	1127	8.57 %	1107	6.65 %	1117	7.61 %	1051	1.25 %	1046	0.77 %
la14	20*5	1070	—	1159	8.32 %	1161	8.50 %	1201	12.24 %	1107	3.46 %	1152	7.66 %	1110	3.74 %	1103	3.08 %
la15	20*5	1090	—	1264	15.96 %	1202	10.28 %	1282	17.61 %	1228	12.66 %	1258	15.41 %	1236	13.39 %	1153	5.78 %
la16	10*10	717	—	913	27.34 %	984	37.24 %	968	35.01 %	794	10.74 %	851	18.69 %	821	14.50 %	786	9.62 %
la17	10*10	646	—	770	19.20 %	794	22.91 %	825	27.71 %	731	13.16 %	892	38.08 %	720	11.46 %	697	7.89 %
la18	10*10	666	—	891	33.78 %	920	38.14 %	888	33.33 %	814	22.22 %	744	11.71 %	766	15.02 %	723	8.56 %
la19	10*10	700	—	882	26.00 %	885	26.43 %	1023	46.14 %	884	26.29 %	952	36.00 %	812	16.00 %	801	14.43 %
la20	10*10	756	—	882	16.67 %	1117	47.75 %	1001	32.41 %	943	24.74 %	964	27.51 %	878	16.14 %	831	9.92 %
la21	15*10	835	—	1055	26.35 %	1096	31.26 %	1146	37.25 %	989	18.44 %	1073	28.50 %	967	15.81 %	966	15.69 %
la22	15*10	760	—	1125	48.03 %	1072	41.05 %	1132	48.95 %	857	12.76 %	1001	31.71 %	893	17.50 %	893	17.50 %
la23	15*10	842	—	1035	22.92 %	1137	35.04 %	1146	36.10 %	956	13.54 %	1066	26.60 %	1010	19.95 %	951	12.95 %
la24	15*10	808	—	1001	23.89 %	1095	35.52 %	1085	34.28 %	958	18.56 %	966	19.55 %	928	14.85 %	913	13.00 %
la25	15*10	791	—	1063	34.39 %	1172	48.17 %	1130	42.86 %	895	13.15 %	964	21.87 %	939	18.71 %	898	13.53 %
la26	20*10	1061	—	1229	15.83 %	1292	21.77 %	1406	32.52 %	1173	10.56 %	1237	16.59 %	1175	10.74 %	1163	9.61 %
la27	20*10	1091	—	1257	15.22 %	1314	20.44 %	1482	35.84 %	1225	12.28 %	1276	16.96 %	1214	11.27 %	1209	10.82 %
la28	20*10	1080	—	1221	13.06 %	1343	24.35 %	1422	31.67 %	1199	11.02 %	1320	22.22 %	1175	8.80 %	1163	7.69 %
la29	20*10	998	—	1250	25.25 %	1387	38.98 %	1273	27.56 %	1101	10.32 %	1219	22.14 %	1111	11.32 %	1080	8.22 %
la30	20*10	1078	—	1230	14.10 %	1361	26.25 %	1382	28.20 %	1227	13.82 %	1371	27.18 %	1275	18.27 %	1193	10.67 %
la31	30*10	1521	—	1644	8.09 %	1829	20.25 %	1840	20.97 %	1591	4.60 %	1624	6.77 %	1636	7.56 %	1569	3.16 %
la32	30*10	1659	—	1824	9.95 %	1913	15.31 %	1931	16.40 %	1798	8.38 %	1849	11.45 %	1765	6.39 %	1735	4.58 %
la33	30*10	1499	—	1605	7.07 %	1891	26.15 %	1700	13.41 %	1568	4.60 %	1663	10.94 %	1542	2.87 %	1559	4.00 %
la34	30*10	1536	—	1702	10.81 %	1747	13.74 %	1767	15.04 %	1577	2.67 %	1599	4.10 %	1686	9.77 %	1577	2.67 %
la35	30*10	1550	—	1737	12.06 %	1773	14.39 %	1764	13.81 %	1692	9.16 %	1706	10.06 %	1706	10.06 %	1629	5.10 %
la36	15*15	1030	—	1320	28.16 %	1372	33.20 %	1411	36.99 %	1296	25.83 %	1296	25.83 %	1229	19.32 %	1162	12.82 %
la37	15*15	1077	—	1299	20.61 %	1488	38.16 %	1528	41.88 %	1210	12.35 %	1285	19.31 %	1294	20.15 %	1177	9.29 %
la38	15*15	962	—	1174	22.04 %	1389	44.39 %	1516	57.59 %	1128	17.26 %	1188	23.49 %	1107	15.07 %	1117	16.11 %
la39	15*15	1024	—	1329	29.79 %	1452	41.80 %	1455	42.09 %	1174	14.65 %	1229	20.02 %	1228	19.92 %	1167	13.96 %
la40	15*15	970	—	1167	20.31 %	1435	47.94 %	1392	43.51 %	1098	13.20 %	1200	23.71 %	1108	14.23 %	1084	11.75 %

Table 13
Comparison of solution quality accross all methods on Hurink (vdata).

Instances	Size	UB		FIFO		MWKR		MOPNR		Song et al. (2023)		Lei et al. (2022)		Yuan et al. (2024)		Ours	
		C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap	C _{max}	Gap
la01	10*5	570	–	665	16.67 %	660	15.79 %	642	12.63 %	609	6.84 %	618	8.42 %	612	7.37 %	590	3.51 %
la02	10*5	529	–	565	6.81 %	662	25.14 %	587	10.96 %	582	10.02 %	583	10.21 %	601	13.61 %	548	3.59 %
la03	10*5	477	–	546	14.47 %	544	14.05 %	595	24.74 %	517	8.39 %	533	11.74 %	533	11.74 %	508	6.50 %
la04	10*5	502	–	527	4.98 %	598	19.12 %	659	31.27 %	554	10.36 %	555	10.56 %	556	10.76 %	525	4.58 %
la05	10*5	457	–	492	7.66 %	578	26.48 %	614	34.35 %	493	7.88 %	563	23.19 %	525	14.88 %	486	6.35 %
la06	15*5	799	–	881	10.26 %	890	11.39 %	938	17.40 %	859	7.51 %	837	4.76 %	888	11.14 %	815	2.00 %
la07	15*5	749	–	810	8.14 %	816	8.95 %	811	8.28 %	785	4.81 %	777	3.74 %	802	7.08 %	762	1.74 %
la08	15*5	765	–	830	8.50 %	848	10.85 %	853	11.50 %	805	5.23 %	838	9.54 %	783	2.35 %	785	2.61 %
la09	15*5	853	–	880	3.17 %	931	9.14 %	939	10.08 %	862	1.06 %	891	4.45 %	887	3.99 %	869	1.88 %
la10	15*5	804	–	837	4.10 %	867	7.84 %	871	8.33 %	839	4.35 %	825	2.61 %	843	4.85 %	814	1.24 %
la11	20*5	1071	–	1135	5.98 %	1153	7.66 %	1117	4.30 %	1114	4.01 %	1120	4.58 %	1119	4.48 %	1076	0.47 %
la12	20*5	936	–	970	3.63 %	1015	8.44 %	1014	8.33 %	967	3.31 %	951	1.60 %	977	4.38 %	941	0.53 %
la13	20*5	1038	–	1078	3.85 %	1084	4.43 %	1136	9.44 %	1076	3.66 %	1072	3.28 %	1082	4.24 %	1043	0.48 %
la14	20*5	1070	–	1083	1.21 %	1113	4.02 %	1119	4.58 %	1099	2.71 %	1115	4.21 %	1088	1.68 %	1074	0.37 %
la15	20*5	1089	–	1122	3.03 %	1111	2.02 %	1152	5.79 %	1104	1.38 %	1154	5.97 %	1110	1.93 %	1098	0.83 %
la16	10*10	717	–	809	12.83 %	767	6.97 %	726	1.26 %	753	5.02 %	768	7.11 %	727	1.39 %	717	0.00 %
la17	10*10	646	–	668	3.41 %	659	2.01 %	728	12.69 %	658	1.86 %	700	8.36 %	648	0.31 %	646	0.00 %
la18	10*10	663	–	671	1.21 %	680	2.56 %	704	6.18 %	663	0.00 %	676	1.96 %	663	0.00 %	663	0.00 %
la19	10*10	617	–	675	9.40 %	647	4.86 %	757	22.69 %	654	6.00 %	697	12.97 %	639	3.57 %	624	1.13 %
la20	10*10	756	–	798	5.56 %	766	1.32 %	807	6.75 %	756	0.00 %	772	2.12 %	766	1.32 %	756	0.00 %
la21	15*10	804	–	920	14.43 %	926	15.17 %	1043	29.73 %	839	4.35 %	894	11.19 %	891	10.82 %	814	1.24 %
la22	15*10	736	–	804	9.24 %	817	11.01 %	857	16.44 %	812	10.33 %	834	13.32 %	809	9.92 %	756	2.72 %
la23	15*10	815	–	871	6.87 %	895	9.82 %	957	17.42 %	868	6.50 %	863	5.89 %	872	6.99 %	833	2.21 %
la24	15*10	775	–	865	11.61 %	879	13.42 %	904	16.65 %	803	3.61 %	872	12.52 %	873	12.65 %	805	3.87 %
la25	15*10	756	–	936	23.81 %	877	16.01 %	926	22.49 %	809	7.01 %	893	18.12 %	840	11.11 %	781	3.31 %
la26	20*10	1054	–	1142	8.35 %	1145	8.63 %	1165	10.53 %	1073	1.80 %	1087	3.13 %	1100	4.36 %	1066	1.14 %
la27	20*10	1084	–	1154	6.46 %	1151	6.18 %	1233	13.75 %	1133	4.52 %	1185	9.32 %	1131	4.34 %	1102	1.66 %
la28	20*10	1070	–	1157	8.13 %	1171	9.44 %	1191	11.31 %	1100	2.80 %	1135	6.07 %	1106	3.36 %	1087	1.59 %
la29	20*10	994	–	1091	9.76 %	1178	18.51 %	1138	14.49 %	1027	3.32 %	1088	9.46 %	1076	8.25 %	1009	1.51 %
la30	20*10	1069	–	1143	6.92 %	1223	14.41 %	1215	13.66 %	1106	3.46 %	1138	6.45 %	1143	6.92 %	1091	2.06 %
la31	30*10	1520	–	1605	5.59 %	1591	4.67 %	1618	6.45 %	1565	2.96 %	1596	5.00 %	1575	3.62 %	1529	0.59 %
la32	30*10	1658	–	1705	2.83 %	1737	4.76 %	1781	7.42 %	1714	3.38 %	1733	4.52 %	1726	4.10 %	1675	1.03 %
la33	30*10	1497	–	1541	2.94 %	1588	6.08 %	1574	5.14 %	1529	2.14 %	1554	3.81 %	1531	2.27 %	1513	1.07 %
la34	30*10	1537	–	1583	2.99 %	1582	2.93 %	1645	7.03 %	1574	2.41 %	1588	3.32 %	1594	3.71 %	1546	0.59 %
la35	30*10	1549	–	1620	4.58 %	1600	3.29 %	1681	8.52 %	1612	4.07 %	1596	3.03 %	1575	1.68 %	1564	0.97 %
la36	15*15	948	–	1015	7.07 %	976	2.95 %	1106	16.67 %	993	4.75 %	964	1.69 %	973	2.64 %	952	0.42 %
la37	15*15	986	–	1082	9.74 %	1006	2.03 %	1155	17.14 %	1069	8.42 %	1066	8.11 %	1024	3.85 %	988	0.20 %
la38	15*15	943	–	952	0.95 %	981	4.03 %	1069	13.36 %	945	0.21 %	979	3.82 %	943	0.00 %	943	0.00 %
la39	15*15	922	–	972	5.42 %	1026	11.28 %	1054	14.32 %	951	3.15 %	984	6.72 %	942	2.17 %	935	1.41 %
la40	15*15	955	–	985	3.14 %	965	1.05 %	1066	11.62 %	985	3.14 %	971	1.68 %	967	1.26 %	955	0.00 %

6. Conclusion and future work

This study has introduced an innovative multi-agent deep reinforcement learning framework that integrates a multi-head self-attention network (MHSAN) with heterogeneous graph embedding for tackling the FJSP. A novel multi-dimensional action and reward Markov decision process (MD-MDP) formulation is proposed, representing the first MDP modeling method specifically designed for FJSPs through multi-action/reward-driven sequential decision model. MD-MDP overcomes limitations in conventional MDP modeling methods by dynamically adjusting decision time to access high-quality action spaces, thereby significantly enhancing scheduling scheme quality for FJSPs. A heterogeneous graph-embedded MHSAN is developed for problem-specific feature representation and extraction in scheduling environments, enabling efficient handling of multi-scale FJSPs through the concatenation and transformation of multi-modal graph features, while capturing critical correlations inherent in the complex constraints of FJSPs. Building upon MD-MDP, the multi-agent MHSAN framework namely MAMHSAN is presented. In MAMHSAN, DDT agent handles decision-time transfers and SAS agent executes joint scheduling actions. This promising paradigm for MD-MDP of FJSPs enables efficient scheduling optimization in FMS through multi-agent coordinated decisions. Furthermore, an efficient PPO-based multi-agent training method is implemented to optimize policy learning, where effective reward shaping and experience sharing accelerate convergence while preserving policy optimality. Extensive experiments on public datasets demonstrate that MAMHSAN has statistically significant superiority in solution quality and exploration efficiency against existing state-of-the-art methods, with robust generalization capabilities in solving FJSPs across diverse scales.

In future work, we will consider using DRL-based methods to solve distributed FJSPs and dynamic FJSPs in real-world production environments. These problems exhibit considerable complexities due to coupling constraints, uncertainties, and dynamic disruptions, thus raising two critical challenges: (1) Traditional heterogeneous graphs and attention networks still suffer from limitations for extracting enriched

features and promising patterns from multi-scale instances, resulting in developing problem-specific graph representation and feature extraction for real-time scheduling in dynamic environments are still worthy of insightful investigations. (2) Typical single-agent DRL frameworks are inadequate for coping with complex and dynamic decision-making requirements, and thus require tailored MDP modeling methods with multi-agent collaborative decisions to ensure efficiency and effectiveness. Addressing these challenges requires pioneering new perspectives in developing innovative feature extraction networks with scale-independent compatibility and multi-agent coordination mechanisms with reward consistency and experience compensation

CRedit authorship contribution statement

Zi-Qi Zhang: Writing – original draft, Methodology, Investigation, Funding acquisition. **Tong Huang:** Writing – original draft, Software, Methodology, Investigation. **Bin Qian:** Writing – review & editing, Methodology, Funding acquisition. **Rong Hu:** Writing – review & editing, Supervision, Methodology, Funding acquisition.

Acknowledgments

The authors are sincerely grateful to the anonymous reviewers for their insightful comments and suggestions, which greatly improve this paper. This work was supported in part by the Youth Project of the National Natural Science Foundation of China (Grant No. 72201115), in part by the National Natural Science Foundation of China (Grant Nos. 72561014, U24A20273, and 62173169), in part by the Yunnan Provincial Outstanding Youth Science Foundation (Grant No. 202501AW070010), in part by Yunnan Fundamental Research Projects (Grant Nos. 202201BE070001-050 and 202301AU070069), in part by the Applied Basic Research Key Project of Yunnan (Grant No. 202201AS070030), and in part by Construction Project of Higher Educational Key Laboratory for Industrial Intelligence and Systems of Yunnan Province (Grant No. KKPH202403003).

Appendix. Definition of raw feature vectors

At each decision time $T(t)$, the features μ_{ij} of operation O_{ij} , features $\xi_{ij,k}$ of original $O - M$ pair, the feature ψ_k of the machine node in the heterogeneous graph and its adjacent node feature $\lambda_{ij,k}$, the feature η_t of the action space, and the feature σ_t of the next decision time are as follows:

For the feature μ_{ij} of the operation O_{ij} , it includes the following content:

μ_{ij}^1 : The release time of the job J_i to which the current operation O_{ij} belongs.

μ_{ij}^2 : The remaining time until the release of the job J_i to which the current operation O_{ij} belongs. If the release time of job J_i is less than or equal to the current decision-making time $T(t)$, then $\mu_{ij}^2 = 0$; otherwise, it is calculated using the formula given by Eq. (A.1).

μ_{ij}^3 : The number of operations remaining to be scheduled for the job J_i to which the current operation O_{ij} belongs.

μ_{ij}^4 : The number of machines available for processing the current operation O_{ij} .

μ_{ij}^5 : The average processing time of the current operation O_{ij} across the machine set M_{ij} available for processing.

μ_{ij}^6 : The minimum processing time of the current operation O_{ij} across the machine set M_{ij} available for processing.

μ_{ij}^7 : The number of machines available for processing the next operation O_{ij+1} in sequence after the current operation O_{ij} .

μ_{ij}^8 : The average processing time of the next operation O_{ij+1} across the machine set M_{ij+1} available for processing.

μ_{ij}^9 : The minimum processing time of the next operation O_{ij+1} across the machine set M_{ij+1} available for processing.

μ_{ij}^{10} : The maximum processing time of the next operation O_{ij+1} across the machine set M_{ij+1} available for processing.

$$\mu_{ij}^2 = T(t) - \mu_{ij}^1 \quad (\text{A.1})$$

$$\psi_k^3 = T(t) - \psi_k^2 \quad (\text{A.2})$$

For the adjacent nodes of the machine M_k in the heterogeneous graph, its features ψ_k include the following:

ψ_k^1 : The number of operations that have been scheduled on the current machine M_k .

ψ_k^2 : The release time of the current machine M_k .

ψ_k^3 : The remaining time until the release of the current machine M_k . If its release time is less than or equal to the current decision time $T(t)$, then

$\psi_k^3 = 0$; otherwise, its calculation formula is given by Eq. (A.2).

For the adjacent operation O_{ij} nodes of the machine M_k in the heterogeneous graph, their feature $\lambda_{ij,k}$ are defined as:

$\lambda_{ij,k}^1$: Whether the operation O_{ij} has been scheduled. If it has been scheduled, the value is 1; otherwise, it is 0.

$\lambda_{ij,k}^2$: The processing time of operation O_{ij} on machine M_k .

$\lambda_{ij,k}^3$: The average processing time of operation O_{ij} across the set of machines M_k available for processing.

$\lambda_{ij,k}^4$: The minimum processing time of operation O_{ij} across the set of machines M_k available for processing.

$$\xi_{ij,k}^2 = \max(\mu_{ij}^1, \psi_k^2) \quad (\text{A.3})$$

$$\xi_{ij,k}^3 = \xi_{ij,k}^2 + \xi_{ij,k}^1 \quad (\text{A.4})$$

$$\xi_{ij,k}^4 = PT_{ij,m} - \min(PT_{ij}) \quad (\text{A.5})$$

For each $O - M$ pair, its raw characteristic $\xi_{ij,k}$ includes the following:

$\xi_{ij,k}^1$: The processing time $PT_{ij,m}$ of the operation O_{ij} in the current combination on machine M_k . If it cannot be processed on machine M_k , the value is 0.

$\xi_{ij,k}^2$: The earliest start time for the operation O_{ij} in the current combination when processed on machine M_k . The calculation method is described in Eq. (A.3).

$\xi_{ij,k}^3$: The earliest completion time for the operation O_{ij} in the current combination when processed on machine M_k . The calculation method is described in Eq. (A.4).

$\xi_{ij,k}^4$: The difference between the minimum processing time and the current processing time of operation O_{ij} . The calculation method is described in Eq. (A.5).

Regarding the definition of the action space characteristic η_T , it is as follows:

η_T^1 : The idle rate of jobs in the current action space.

η_T^2 : At decision time $T(t)$, the average release time of jobs in the action space.

η_T^3 : At decision time $T(t)$, the idle rate of machines in the action space.

η_T^4 : At decision time $T(t)$, the average release time of machines in the action space.

η_T^5 : At decision time $T(t)$, among the valid actions in the action space, the ratio of the processing time of O_{ij} on M_k to the shortest processing time of O_{ij} .

η_T^6 : At decision time $T(t)$, the ratio of all actions in the action space that can achieve a shorter processing time at the next decision time $T'(t)$.

η_T^7 : At decision time $T(t)$, the ratio of all actions in the action space that can achieve the shortest processing time at the next decision time $T'(t)$.

Regarding the characteristics of the next decision time $T'(t)$, it is defined as follows:

σ_T : The time difference between the current decision time $T(t)$ and the next decision time $T'(t)$, the calculation formula is given by Eq. (A.6).

$$\sigma_T = T(t) - T'(t) \quad (\text{A.6})$$

Data availability

Data will be made available on request.

References

- Caldeira, R. H., & Gnanavelbabu, A. (2019). Solving the flexible job shop scheduling problem using an improved Jaya algorithm. *Computers & Industrial Engineering*, 137, Article 106064. <https://doi.org/10.1016/j.cie.2019.106064>
- Chen, R., Yang, B., Li, S., & Wang, S. (2020). A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering*, 149, Article 106778. <https://doi.org/10.1016/j.cie.2020.106778>
- Chen, R., Li, W., & Yang, H. (2023). A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job-shop scheduling problem. *IEEE Transactions on Industrial Informatics*, 19, 1322–1331. <https://doi.org/10.1109/TII.2022.3167380>
- Chiang, T. C., & Fu, L. C. (2007). Using dispatching rules for job shop scheduling with due date-based objectives. *International Journal of Production Research*, 45, 3245–3262. <https://doi.org/10.1080/00207540600786715>
- Cunha, B., Madureira, A. M., Fonseca, B., & Coelho, D. (2018). (Year). deep reinforcement learning as a job shop scheduling solver: A literature review. In *Hybrid Intelligent Systems: 18th International Conference on Hybrid Intelligent Systems (HIS 2018)* (pp. 350–359). https://doi.org/10.1007/978-3-030-14347-3_34
- Demir, Y., & Kürşat İşleyen, S. (2013). Evaluation of mathematical models for flexible job-shop scheduling problems. *Applied Mathematical Modelling*, 37, 977–988. <https://doi.org/10.1016/j.apm.2012.03.020>
- Doh, H.-H., Yu, J.-M., Kim, J.-S., Lee, D.-H., & Nam, S.-H. (2013). A priority scheduling approach for flexible job shops with multiple process plans. *International Journal of Production Research*, 51, 3748–3764. <https://doi.org/10.1080/00207543.2013.765074>
- Gran, S.S., Ismail, I., Ajol, T.A. & Ibrahim, A.F.A. (Year). Mixed Integer programming model for flexible job-shop scheduling problem (FJSP) to minimize makespan and total machining time. *2015 International Conference on Computer, Communications, and Control Technology (I4CT)*, 413–417. doi:10.1109/I4CT.2015.7219609.
- Guo, M.-H., Xu, T.-X., Liu, J.-J., Liu, Z.-N., Jiang, P.-T., Mu, T.-J., Zhang, S.-H., Martin, R. R., Cheng, M.-M., & Hu, S.-M. (2022). Attention mechanisms in computer vision: A survey. *Computational Visual Media*, 8, 331–368. <https://doi.org/10.1007/s41095-022-0271-y>
- Han, X., Cheng, W., Meng, L., Zhang, B., Gao, K., Zhang, C., & Duan, P. (2024). A dual population collaborative genetic algorithm for solving flexible job shop scheduling problem with AGV. *Swarm and Evolutionary Computation*, 86, Article 101538. <https://doi.org/10.1016/j.swevo.2024.101538>
- Han, K., & Gong, W. (2025). Memetic algorithm based on non-dominated levels for flexible job shop scheduling problem with learn-forgetting effect and worker cooperation. *Computers & Industrial Engineering*, 200, Article 110845. <https://doi.org/10.1016/j.cie.2024.110845>
- Haupt, R. (1989). A survey of priority rule-based scheduling. *Operations-Research-Spektrum*, 11, 3–16. <https://doi.org/10.1007/BF01721162>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444. <https://doi.org/10.1038/nature14539>
- Lei, K., Guo, P., Zhao, W., Wang, Y., Qian, L., Meng, X., & Tang, L. (2022). A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Systems with Applications*, 205, Article 117796. <https://doi.org/10.1016/j.eswa.2022.117796>
- Lei, K., Guo, P., Wang, Y., Zhang, J., Meng, X., & Qian, L. (2024). Large-scale dynamic scheduling for flexible job-shop with random arrivals of new jobs by hierarchical reinforcement learning. *IEEE Transactions on Industrial Informatics*, 20, 1007–1018. <https://doi.org/10.1109/TII.2023.3272661>
- Li, X., Guo, X., Tang, H., Wu, R., Wang, L., Pang, S., Liu, Z., Xu, W., & Li, X. (2022). Survey of integrated flexible job shop scheduling problems. *Computers Industrial Engineering*, 174, Article 108786.

- Li, C., Zheng, P., Yin, Y., Wang, B., Wang, L., & Technology. (2023). Deep reinforcement learning in smart manufacturing: A review and prospects. *Journal of Manufacturing Science*, 40, 75–101. <https://doi.org/10.1016/j.cirpj.2022.11.003>
- Liu, R., Piplani, R., & Toro, C. (2022). Deep reinforcement learning for dynamic scheduling of a flexible job shop. *International Journal of Production Research*, 60, 4049–4069. <https://doi.org/10.1080/00207543.2022.2058432>
- Liu, R., Piplani, R., & Toro, C. (2023). A deep multi-agent reinforcement learning approach to solve dynamic job shop scheduling problem. *Computers Operations Research*, 159, Article 106294. <https://doi.org/10.1016/j.cor.2023.106294>
- Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91, Article 106208. <https://doi.org/10.1016/j.asoc.2020.106208>
- Luo, Q., Deng, Q., Xie, G., & Gong, G. (2023). A Pareto-based two-stage evolutionary algorithm for flexible job shop scheduling problem with worker cooperation flexibility. *Robotics Computer-Integrated Manufacturing*, 82, Article 102534. <https://doi.org/10.1016/j.rcim.2023.102534>
- Luo, S., Zhang, L., & Fan, Y. (2022). Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning. *IEEE Transactions on Automation Science and Engineering*, 19, 3020–3038. <https://doi.org/10.1109/TASE.2021.3104716>
- Meng, L., Zhang, C., Shao, X., & Ren, Y. (2019). MILP models for energy-aware flexible job shop scheduling problem. *Journal of Cleaner Production*, 210, 710–723. <https://doi.org/10.1016/j.jclepro.2018.11.021>
- Meng, L., Zhang, C., Shao, X., Zhang, B., Ren, Y., & Lin, W. (2020). More MILP models for hybrid flow shop scheduling problem and its extended problems. *International Journal of Production Research*, 58, 3905–3930. <https://doi.org/10.1080/00207543.2019.1636324>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533. <https://doi.org/10.1038/nature14236>
- Munikoti, S., Agarwal, D., Das, L., Halappanavar, M., & Natarajan, B. (2024). Challenges and opportunities in deep reinforcement learning with graph neural networks: A comprehensive review of algorithms and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 35, 15051–15071. <https://doi.org/10.1109/TNNLS.2023.3283523>
- Naderi, B., & Roshanaei, V. (2022). Critical-path-search logic-based benders decomposition approaches for flexible job shop scheduling. *INFORMS Journal on Optimization*, 4, 1–28. <https://doi.org/10.1287/ijoo.2021.0056>
- Narassima, M. S., Vasudevan, S. K., & Anbudayasankar, S. P. (2022). Development of a priority dispatching rule and novel production scheduling algorithm for a job shop. *International Journal of Computer Aided Engineering and Technology*, 16, 417–449. <https://doi.org/10.1504/IJCAET.2022.123989>
- Pan, Z., Wang, L., Wang, J., & Lu, J. (2023). Deep reinforcement learning based optimization algorithm for permutation flow-shop scheduling. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 7, 983–994. <https://doi.org/10.1109/TETCI.2021.3098354>
- Panzer, M., & Bender, B. (2022). Deep reinforcement learning in production systems: A systematic literature review. *International Journal of Production Research*, 60, 4316–4341. <https://doi.org/10.1080/00207543.2021.1973138>
- Park, J., Chun, J., Kim, S. H., Kim, Y., & Park, J. (2021). Learning to schedule job-shop problems: Representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59, 3360–3377. <https://doi.org/10.1080/00207543.2020.1870013>
- Rooyani, D., & Defersha, F. M. (2019). An Efficient Two-Stage Genetic Algorithm for Flexible Job-Shop Scheduling. *IFAC-PapersOnLine*, 52, 2519–2524. <https://doi.org/10.1016/j.ifacol.2019.11.585>
- Sels, V., Gheysen, N., & Vanhoucke, M. (2012). A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. *International Journal of Production Research*, 50, 4255–4270. <https://doi.org/10.1080/00207543.2011.611539>
- Sharma, V., & Tripathi, A. K. (2022). A systematic review of meta-heuristic algorithms in IoT based application. *Array*, 14, Article 100164. <https://doi.org/10.1016/j.array.2022.100164>
- Sobeyko, O., & Mönch, L. (2016). Heuristic approaches for scheduling jobs in large-scale flexible job shops. *Computers & Operations Research*, 68, 97–109. <https://doi.org/10.1016/j.cor.2015.11.004>
- Song, W., Chen, X., Li, Q., & Cao, Z. (2023). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 19, 1600–1610. <https://doi.org/10.1109/TII.2022.3189725>
- Sun, K., Zheng, D., Song, H., Cheng, Z., Lang, X., Yuan, W., & Wang, J. (2023). Hybrid genetic algorithm with variable neighborhood search for flexible job shop scheduling problem in a machining system. *Expert Systems with Applications*, 215, Article 119359. <https://doi.org/10.1016/j.eswa.2022.119359>
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT Press, 17, 229–235. <https://doi.org/10.1017/S0263574799271172>
- Thenarasu, M., Rameshkumar, K., Rousseau, J., & Anbudayasankar, S. P. (2022). Development and analysis of priority decision rules using MCDM approach for a flexible job shop scheduling: A simulation study. *Simulation Modelling Practice and Theory*, 114, Article 102416. <https://doi.org/10.1016/j.simpat.2021.102416>
- Vaswani, A. A. S., Noam, Niki, Parmar, Jakob, Uszkoreit, Jones Llion, N., Aidan, Gomez, Ukasz, Kaiser, & Illia, Polosukhin (2017). Attention is all you need. *Curran Associates*, 30. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- Wan, L., Fu, L., Li, C., & Li, K. (2024). Flexible job shop scheduling via deep reinforcement learning with meta-path-based heterogeneous graph neural network. *Knowledge-Based Systems*, 296, Article 111940. <https://doi.org/10.1016/j.knsys.2024.111940>
- Wan, L., Fu, L., Li, C., & Li, K. (2025). An effective multi-agent-based graph reinforcement learning method for solving flexible job shop scheduling problem. *Engineering Applications of Artificial Intelligence*, 139, Article 109557. <https://doi.org/10.1016/j.engappai.2024.109557>
- Wang, S., & Zhang, H. (2023). A matheuristic for flowshop scheduling with batch processing machines in textile manufacturing. *Applied Soft Computing*, 145, Article 110594. <https://doi.org/10.1016/j.asoc.2023.110594>
- Witty, S., Lee, J. K., Tosch, E., Atrey, A., Clary, K., Littman, M. L., & Jensen, D. (2021). Measuring and characterizing generalization in deep reinforcement learning. *Applied AI Letters*, 2, 45. <https://doi.org/10.1002/ail2.45>
- Wu, K., Huang, E., Wang, M., & Zheng, M. (2022). Job scheduling of diffusion furnaces in semiconductor fabrication facilities. *European Journal of Operational Research*, 301, 141–152. <https://doi.org/10.1016/j.ejor.2021.09.044>
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2020). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32, 4–24. <https://doi.org/10.1109/TNNLS.2020.2978386>
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2021). A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32, 4–24. <https://doi.org/10.1109/TNNLS.2020.2978386>
- Xie, J., Gao, L., Peng, K., Li, X., & Li, H. (2019). Review on flexible job shop scheduling. *IET collaborative intelligent manufacturing*, 1, 67–77. <https://doi.org/10.1049/iet-cim.2018.0009>
- Xu, Y., Zhang, M., Yang, M., & Wang, D. (2024). Hybrid quantum particle swarm optimization and variable neighborhood search for flexible job-shop scheduling problem. *Journal of Manufacturing Systems*, 73, 334–348. <https://doi.org/10.1016/j.jmsy.2024.02.007>
- Yang, Y., Song, Y., Guo, W., Lei, Q., Sun, A., & Fan, L. (2023). Guided shuffled frog-leaping algorithm for flexible job shop scheduling problem with variable sublots and overlapping in operations. *Computers Industrial Engineering*, 180, Article 109209. <https://doi.org/10.1016/j.cie.2023.109209>
- Yang, D., Wu, M., Li, D., Xu, Y., Zhou, X., & Yang, Z. (2022). Dynamic opposite learning enhanced dragonfly algorithm for solving large-scale flexible job shop scheduling problem. *Knowledge-Based Systems*, 238, Article 107815. <https://doi.org/10.1016/j.knsys.2021.107815>
- Yang, T., Yi, X., Lu, S., Johansson, K. H., & Chai, T. (2021). Intelligent manufacturing for the process industry driven by industrial artificial intelligence. *Engineering*, 7, 1224–1230. <https://doi.org/10.1016/j.eng.2021.04.023>
- Yuan, E., Wang, L., Cheng, S., Song, S., Fan, W., & Li, Y. (2024). Solving flexible job shop scheduling problems via deep reinforcement learning. *Expert Systems with Applications*, 245, Article 123019. <https://doi.org/10.1016/j.eswa.2023.123019>
- Zeng, Z., Li, X., & Bai, C. (Year). A Deep Reinforcement Learning Approach to Flexible Job Shop Scheduling. *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 884–890. doi:10.1109/SMC53654.2022.9945107.
- Zhang, L., Feng, Y., Xiao, Q., Xu, Y., Li, D., Yang, D., & Yang, Z. (2023a). Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times. *Journal of Manufacturing Systems*, 71, 257–273. <https://doi.org/10.1016/j.jmsy.2023.09.009>
- Zhang, Z.-Q., Li, Y., Qian, B., Hu, R., & Yang, J.-B. (2024b). A multidimensional probabilistic model based evolutionary algorithm for the energy-efficient distributed flexible job-shop scheduling problem. *Engineering Applications of Artificial Intelligence*, 135, Article 108841. <https://doi.org/10.1016/j.engappai.2024.108841>
- Zhang, Z.-Q., Qiu, X.-H., Qian, B., Hu, R., Wang, L., & Yang, J.-B. (2025). Knowledge-enhanced multidimensional estimation of distribution hyper-heuristic evolutionary algorithm for semiconductor final testing scheduling problem. *Expert Systems with Applications*, 260, Article 125424. <https://doi.org/10.1016/j.engappai.2024.108841>
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Chi, X. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in Neural Information Processing Systems*, 33, 1621–1632. https://proceedings.neurips.cc/paper_files/paper/2020/file/11958dfee29b6709f48a9ba0387a2431-Paper.pdf
- Zhang, Z.-Q., Wu, F.-C., Qian, B., Hu, R., Wang, L., & Jin, H.-P. (2023c). A Q-learning-based hyper-heuristic evolutionary algorithm for the distributed flexible job-shop scheduling problem with crane transportation. *Expert Systems with Applications*, 234, Article 121050. <https://doi.org/10.1016/j.eswa.2023.121050>
- Zhang, W., Zheng, Y., & Ahmad, R. (2023b). An energy-efficient multi-objective scheduling for flexible job-shop-type remanufacturing system. *Journal of Manufacturing Systems*, 66, 211–232. <https://doi.org/10.1016/j.jmsy.2022.12.008>
- Zhang, W., Zhao, F., Li, Y., Du, C., Feng, X., & Mei, X. (2024a). A novel collaborative agent reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for flexible job shop scheduling problem. *Journal of Manufacturing Systems*, 74, 329–345. <https://doi.org/10.1016/j.jmsy.2024.03.012>
- Zhao, F., He, X., & Wang, L. (2021). A two-stage cooperative evolutionary algorithm with problem-specific knowledge for energy-efficient scheduling of no-wait flow-shop problem. *IEEE Transactions on Cybernetics*, 51, 5291–5303. <https://www.ncbi.nlm.nih.gov/pubmed/33095728>